

ManyRVData Project Report

**CachePool: Manycore Cluster of Customizable, Lightweight
Scalar-Vector PEs for Irregular Data-Plane Workloads**



Report by:

Diyoun Shen, Ph.D. Student
Zexin Fu, Ph.D. Student
dishen, zexifu@iis.ee.ethz.ch

Supervised by:

Prof. Luca Benini
Prof. Alessandro Vanelli-Coralli
lbenini, avanelli@iis.ee.ethz.ch

Integrated Systems Laboratory,
ETH Zürich,
Zürich, Switzerland

October 6, 2026

Contents

1	Executive Summary	4
2	Introduction	6
2.1	State of the Arts	6
2.1.1	On-Chip Memory Organization in Manycore Architectures	6
2.1.2	Cache Coherence Schemes	7
2.1.3	On-Chip Interconnection	8
2.2	Project Objectives, Design Philosophy, and Contributions	8
3	CachePool Architecture	12
3.1	System Overview	12
3.2	Core Complex	13
3.2.1	Snitch Scalar Core	13
3.2.2	Spatz Vector Coprocessor	13
3.2.3	Core-Complex Configurations	13
3.2.4	Core-Local Stack Memory	14
3.3	Tile Architecture	14
3.3.1	Tile Organization	14
3.3.2	Local L1 Access Network	14
3.3.3	Atomic Operations	15
3.4	Group Architecture	16
3.5	Distributed and Partitionable L1 Data Cache	16
3.5.1	Global Shared-L1 Organization	16
3.5.2	Address Mapping and Software-Controlled Scrambling	16
3.5.3	Cache Partitioning	17
3.6	L1 Cache Controller	17
3.6.1	Request Coalescer	17
3.6.2	In-Situ Cache Overview	18
3.6.3	Unified In-Cacheline Miss Handling	18
3.6.4	Request and Refill Pipelines	19
3.6.5	Way Prediction and Replacement	19
3.6.6	Folded Cache Organization	20
3.7	Hierarchical Load/Store Interconnect	20
3.7.1	Interconnect Organization and Protocol	20
3.7.2	Intra-Tile Access	20
3.7.3	Intra-Group Access	20
3.7.4	Inter-Group Narrow Meshes	21
3.7.5	Non-Geometric Destination Routing	22
3.7.6	L1 Network Latency	22
3.8	Cache Refill and Main-Memory Network	23
3.8.1	Wide Refill Mesh	23
3.8.2	HBM Connectivity	24
3.8.3	Refill Routing and Protocol Conversion	24
3.9	Instruction Memory Hierarchy	26
3.9.1	Per-Core L0 Instruction Cache	26
3.9.2	Tile-Shared L1 Instruction Cache	26
3.9.3	Group-Shared L2 Instruction Cache	26
3.10	System Control and Synchronization	26
3.10.1	Memory-Mapped Control and Status Registers	26
3.10.2	Hardware Barrier	27

3.11	System Configuration and Scalability	27
3.12	DRAM/HBM Co-Simulation	27
4	Physical Implementation	31
4.1	Methodology	31
4.2	Floorplan	31
4.3	Area Breakdown	31
5	Software and Runtime Support	34
5.1	Programming Model	34
5.2	Topology Discovery and Core Identification	34
5.3	Memory Allocation and Placement	35
5.3.1	L1 Address Scrambling	35
5.3.2	Cache Partitioning and Address Classification	35
5.3.3	Cache Maintenance	36
5.4	Synchronization and Work Distribution	36
5.4.1	Memory Ordering and Fences	36
5.4.2	Hardware and Software Barriers	36
5.4.3	Atomic Operations	37
5.4.4	Work Distribution	37
5.5	Spatz Control	37
5.6	Boot, Execution, and Profiling Support	38
6	GVSoc Simulation Framework	39
6.1	CachePool System Model	39
6.2	Configurable Architecture Exploration	39
6.3	Memory-System Modeling	39
6.4	Software Compatibility and End-to-End Simulation	40
6.5	Model Calibration and RTL Validation	40
6.6	Current Limitations	40
7	Kernel Characterization	41
7.1	Dot Product (DOTP)	41
7.2	Matrix-Vector Multiplication (MVM)	41
7.3	Matrix Multiplication (MatMul)	41
7.4	Shannon–Hartley Channel Capacity Formula (Shannon)	42
7.5	Vector Sorting	42
8	RLC/MAC Dataplane Implementation	43
8.1	Workload model and constraints	43
8.2	Software Architecture and Implementation	43
8.2.1	Workflow of the Radio Link Control (RLC) Kernel	43
8.2.2	Producer–Consumer Roles	44
8.2.3	Concurrency and Synchronization	45
8.2.4	Data Structures and Memory Management	45
8.2.5	Task Scheduling and Thread Management	45
9	Experimental Methodology	46
9.1	Evaluated System Configurations	46
9.2	Simulation and Memory-System Setup	46
9.3	Benchmarks and Metrics	46

10 Evaluation	46
10.1 Vector Kernel Benchmarking	46
10.2 RLC Kernel Benchmarking	49
11 Discussion	51
12 Future Work	51
12.1 Data Cache Optimization	51
12.2 Data Cache Partition Exploration	51
12.3 Processing Element Microarchitecture Optimization	51
12.4 Multi-Tile Scaling and Interconnect Exploration	51
12.5 Advanced Backend Implementation and PPA Evaluation	52
12.6 Software Kernel Finalization and Evaluation	52
12.7 Exploration of Private L1 Cache	52
13 Conclusion	53
A Private Write-Through L0.5 Cache Exploration	57
A.1 Motivation and Scope	57
A.2 Hardware-Coherent Exploration	57
A.3 Software-Managed Coherence	57
A.3.1 Private L0.5 Organization	58
A.3.2 Synchronization and Data Visibility	58
A.3.3 Runtime Cache-Management Operations	58
A.3.4 Kernel Integration	59
A.4 Preliminary Performance Observations	59
A.5 Discussion and Future Work	60
B CachePool Runtime API Reference	61
C Spatz Supported Vector Instructions	67

1 Executive Summary

This document reports on the *ManyRVData* project. The aim of the project is to design and evaluate a scalable, cache-based manycore architecture with scalar-vector capability for the 5G Radio Access Networks (RAN) dataplane. The focus is on efficiently supporting the RLC layer, which requires both high-throughput packet processing and control parameter computation under tight latency constraints. The architecture is designed to maintain high performance and energy efficiency when scaling to hundreds of Processing Elements (PEs), each operating on large, sparse, and irregular data structures typical of dataplane workloads.

The project explores a heterogeneous manycore cluster in which lightweight, in-order scalar RISC-V cores handle control-intensive tasks, while vector engines excel at data-intensive workloads, all connected through a shared, cache-based memory hierarchy. To better match the unpredictable memory access patterns in RLC packet processing, the design abandons scratchpad-only approaches (such as in MemPool) and avoids hardware cache coherence entirely. Instead, it employs a **shared L1 cache with configurable private L1 partitions** per PE or per group of PEs, enabling flexibility in bandwidth allocation and minimizing contention while keeping implementation complexity low.

The architecture is designed to address three representative RLC dataplane scenarios. The first is a **single-user peak-rate case**, representing the extreme throughput demand of a single access user and serving as a stress test for per-user performance, latency, and sustained packet processing under continuous load. The second is a **multi-user peak-load case**, where a moderate number of active users generate high aggregate throughput, testing the system's ability to scale across many threads while maintaining low contention and predictable latency. The third is a **multi-user average-load case**, reflecting a large-scale deployment with thousands of concurrent users at more typical per-user rates, emphasizing fairness, resource isolation, and stable long-term operation. In the first year, the project will concentrate on the single-user peak-rate case to validate the core architecture, memory subsystem, and scalar-vector processing efficiency before extending evaluation to multi-user scenarios in the second stage.

The project spans two years, with the first-year activities organized into three main Work Packages (WPs):

- **WP1 – Scalable Cache Design for a Many-core Cluster:** Define the architecture specification and explore design options, followed by the initial design of key building blocks. Carry out preliminary implementation and power–performance–area (PPA) analysis.
- **WP2 – Processing Element: Microarchitecture and ISA Optimization:** Specify the microarchitecture and prototype a non-blocking load/store interface, together with instruction set extensions (vector unit) for dataplane processing workloads and control parameter computation.
- **WP3 – HW-SW Mapping, Runtime Management, and Benchmarking:** Conduct preliminary benchmarking with representative dataplane packet-processing kernels, and develop prototype programming APIs and runtime support for deployment, allocation, and scheduling.

Building on ETH Zürich's experience with the *MemPool* architecture, the *ManyRVData* design transitions from software-managed scratchpads to a hardware-managed, shared cache system, while keeping the per-core architecture simple and scalable. This approach provides high flexibility in resource partitioning and efficient support for sparse, irregular workloads without the area and energy costs of full hardware coherence.

The first year will deliver the following milestones:

- **M1.1 – Preliminary ManyRVData Cluster Design (M12):** Release of open-source hardware and documentation.

- **M1.2 – Preliminary Benchmark Suite and Analysis (M12):** Release of open-source software and documentation, accompanied by a benchmarking report.

TODO: Change to full WP

The design source code for hardware and software will be made available as open source.

2 Introduction

Wireless baseband dataplane processing is a prime domain for energy-efficient parallel computing: the RAN market is vast, and the throughput and efficiency requirements are escalating. As 5G evolves toward 5.5G and 6G, per-cell computational complexity is projected to grow far beyond what technology scaling alone can deliver, demanding architectural and implementation advances across the stack. This project addresses that challenge with a focus on dataplane tasks in the RLC layer. Service Data Units (SDUs) from higher layers are queued and assembled into Protocol Data Units (PDUs) with sequence headers for retransmission management before being passed to MAC, an interaction that tightly couples performance and reliability constraints.

Viewed as a computing workload, RLC exhibits three defining features. First, scale: modern deployments must sustain on the order of 10 million packets/s overall, with each RLC instance exceeding 10^3 packets/s and thousands of instances active concurrently. Second, latency: the Transition Time Interval (TTI) spans $62.5\ \mu\text{s}$ to $1\ \text{ms}$ (typically $500\ \mu\text{s}$), setting a hard real-time envelope for all sub-tasks. Third, irregularity: per-user tasks are independent but highly variable, the code footprint is small, while data structures (e.g., linked lists) create sparse, non-contiguous accesses over large address ranges. Together, these properties imply hundreds of concurrent threads touching dispersed state with poor spatial locality.

To ground our design choices, the remainder of this section surveys state-of-the-art manycore architectures alongside prevailing cache-coherence strategies and evaluates their feasibility under the RLC workload’s stringent throughput, TTI-scale latency, and irregular memory behavior. We then introduce the high-level design of *ManyRVData* and summarize its principal contributions.

2.1 State of the Arts

2.1.1 On-Chip Memory Organization in Manycore Architectures

Modern manycore processors use either hardware-managed caches or software-managed scratchpads as their L1 memory. Scratchpad designs give each core or tile a shared banked memory without coherence. For example, the MemPool architecture is a manycore architecture with hundreds of individually programmable cores sharing a low-latency L1 Scratchpad Memory (SPM) [1] and TeraPool scales further beyond a thousand cores with a shared SPM [2]. Such systems achieve very low and predictable global access latency (e.g. 5 cycles) but require explicit Direct Memory Access (DMA) transfers and data placement by software.

In contrast, some other manycore designs use hardware-managed caches. In general purpose CPUs, recent systems include *AmpereOne* (96–192 Arm cores) with 64 KB L1D and 2 MB private L2 per core plus a 64 MB system-level cache on a mesh interconnect [3]; AMD’s 128-core *EPYC Bergamo*, which composes eight 16-core CCDs around shared L3 slices for a large last-level cache [4]; and Intel’s *Sierra Forest* Xeon, which tiles E-cores into clusters that share L2/L3 slices over a mesh fabric [5]. These manycore CPU designs target control-flow-heavy code with irregular and discontinuous memory access. Compared to compute-intensive accelerators, cache hierarchies offer better performance for workloads with unpredictable data access and potential for temporal locality and reuse. In GPGPUs, hardware-managed caches are also employed to improve programmability and performance under diverse workloads. NVIDIA’s Hopper GPUs, for example, feature a unified per-SM memory block that serves as both L1 cache and shared memory [6]. This physical memory can be partitioned at runtime: one portion operates as a hardware-managed L1 cache, while the other functions as software-managed shared memory. This configurable design allows the architecture to adapt to both regular, contiguous data access patterns (where shared memory excels), and irregular, unpredictable access patterns (where L1 caching provides robustness without manual data management). It is also very inspiring to configure the same block of memory into different management schemes.

In summary, dataflow processors and ASIC-based accelerators tend to favor scratchpads and

static scheduling for their highly predictable access patterns, whereas general-purpose CPUs and GPUs utilize hardware-managed caches to handle control and irregular workloads more efficiently. Inspired by these architectural trade-offs, our project targets the RLC workload, which involves frequent control flow, linked-list traversal, pointer chasing, and synchronization mechanisms such as spin locks. These characteristics result in dynamic and unpredictable memory access patterns that are better served by a cache-based memory hierarchy. Therefore, we adopt a hardware-managed cache design as the L1 memory.

2.1.2 Cache Coherence Schemes

Traditional CPU coherence. Conventional multiprocessors maintain cache-line coherence system-wide via snooping (broadcast on a shared bus/ring) or directory-based protocols. While effective up to tens of cores, both approaches encounter fundamental scaling barriers in manycore, data-intensive settings: snooping saturates interconnect bandwidth and power with broadcast traffic; directories incur metadata that grows with sharers (and indirection latency), plus invalidation/ownership traffic that contends on the NoC under fine-grained sharing and write bursts. These effects manifest as coherence hotspots, rising miss/ack latencies, and energy overheads that compete with useful bandwidth, motivating alternative schemes in emerging designs [7, 8, 9].

Emerging schemes for heterogeneous and manycore systems. To accommodate accelerators and extreme parallelism, several directions relax granularity, timing, or scope. (i) *Coarse-grained (region) coherence* amortizes metadata/traffic by tracking permissions over larger regions, particularly effective for CPU–GPU sharing where data moves in bulk [10, 11]. (ii) *Timestamp-based coherence* (e.g., Tardis) replaces invalidations with logical time, eliminating sharer vectors and broadcasts while retaining performance comparable to directories on large core counts [9, 12]. (iii) *Scoped/domain-limited coherence* confines coherence to domains to cap protocol traffic and metadata. In ARM systems, shareability attributes define Non-shareable, Inner Shareable, and Outer Shareable domains; ACE/CHI interconnects and CMN-class meshes honor these domains for coherence snoop and maintenance behavior, enabling cluster-local coherence while avoiding system-wide broadcast [13, 14, 15]. HSA formalizes a complementary scoped memory model (work-group/agent/system) so software can request visibility at the smallest required scope [16, 17]. At the host–device boundary, CXL-cache and CCIX protocols create link-local coherent domains between CPUs and attached accelerators, maintaining coherence only within the CXL/CCIX fabric rather than the entire platform [18, 19, 20, 21]. These designs exemplify domain scoping: keep strong coherence where it pays (clusters, coherent links), and rely on software/fences when crossing domains.

Software-managed or no-coherence designs. Several research/industrial manycores deliberately omit global coherence, shifting responsibility to software/runtimes. Cerebras’ Swarm/SwarmX interconnects all cores and systems for software-managed data movement without traditional caches or global hardware coherence [22]. Many RISC-V accelerator clusters (e.g., Mem-Pool/TeraPool) adopt a shared, multi-banked L1 SPM (Tightly Coupled Data Memory (TCDM)) and rely on DMA and software synchronization rather than cache protocols [1, 23]. Intel’s 48-core Single-Chip Cloud Computer (SCC) deliberately omits coherence; coherence is maintained via message passing and explicit cache management [24]. Industrial manycores such as Kalray MPPA organize cores into clusters that share a local multi-bank memory with software-managed coherence and no global coherence domain [25]. In the accelerator space, GPUs historically employ scoped visibility with fences and atomics; recent work [26] explores extending coherence across hierarchical multi-GPU systems without full broadcast. These approaches reduce hardware complexity and traffic, but shift responsibility for correctness and data movement onto the runtime, an attractive trade-off when per-flow state is private and communication is structured.

The target RLC workload involves linked list traversal, branch-heavy control logic, and frequent synchronization using spinlocks and atomics, all within tight TTI-scale deadlines. In this project, we design the L1 memory to be cache-based to handle irregular and sparse memory access. We further simplify the cache coherence design by making all L1 cache partitions shared by default. At the same time, when certain data structures (e.g., local stack, per-user buffers or hot data) are accessed heavily by a single tile and are sensitive to contention, the system allows configuring selected L1 partitions as private to each tile, with software responsible for maintaining cache coherence. Carefully partitioning shared and private regions helps reduce interference while preserving flexibility and programmability.

2.1.3 On-Chip Interconnection

The on-chip interconnection is a critical component in connecting PEs with memory subsystems in a cluster. Existing approaches can be broadly categorized into (1) low-latency crossbars (Xbars) and (2) router-linked meshes.

Xbars provide low-latency access by enabling direct connections between all endpoints, but their scalability is limited by quadratic growth in routing complexity and wiring overhead. Multi-level memory hierarchies can act as coalescers to reduce the number of Xbar endpoints [27], but at the cost of increased latency and data duplication across hierarchy levels. Hierarchical Xbars also help relieve routing pressure, but the effective bandwidth per core decreases as the system scales [28, 29]. Software techniques such as task scheduling [30, 31] and dynamic address reallocation [32] can reduce interconnect pressure, but they cannot fundamentally overcome the bandwidth limitations of Xbar-based designs and impose additional programming complexity.

Meshes, by contrast, offer better scalability, with latency increasing linearly in the number of hops. The *Celerity* manycore system [33] employs a 2D-mesh Network on Chip (NoC) with a Partitioned Global Address Space (PGAS) memory system to connect 496 cores, but relies heavily on software to tolerate the high communication latency, complicating programmability. The *HammerBlade* architecture [34] uses a half-Ruche NoC [35] to connect cores and cache banks, but still struggles to sustain non-blocking memory accesses at scale, even with support for up to 63 outstanding requests.

More recent designs attempt to combine the low-latency of Xbars with the scalability of meshes through hybrid interconnection schemes. TensTorrent’s *Wormhole* architecture [36] uses Xbars locally to connect five PEs and memory banks within a cluster, and a router-linked 2D-torus to connect 80 clusters globally. Similarly, *ET-SoC-1* [37] clusters 32 cores using Xbars and scales further to 1088 cores through a 2D-mesh NoC. The *TeraNoc* architecture [38] follows a comparable approach, replacing the hierarchical Xbars in *TeraPool* with a hybrid Xbar–NoC interconnect to balance latency and scalability.

Overall, interconnection fabrics in manycore systems expose a fundamental trade-off between latency, scalability, and programmability. Xbars deliver low-latency access but face quadratic growth in complexity, meshes provide scalable layouts but incur higher and variable latency, and hybrid designs attempt to balance the two. While software techniques can mitigate some of these challenges, they increase programming complexity and do not fundamentally resolve the bandwidth limitations. This underscores the need for interconnect solutions that sustain high utilization at scale without prohibitive hardware or software overhead.

2.2 Project Objectives, Design Philosophy, and Contributions

The *ManyRVData* project targets scalable manycore processing for irregular and synchronization-intensive dataplane workloads. These workloads combine fine-grained scalar control, data-parallel computation, sparse and pointer-based memory accesses, and frequent communication among many concurrent processing elements. Such access patterns are difficult to serve efficiently with explicitly managed scratchpad-memory hierarchies, where software techniques such as tiling and

double buffering rely on regular and predictable data movement. At the same time, conventional private-cache hierarchies introduce substantial coherence and interconnect complexity when scaled to hundreds of processing elements.

To address these challenges, ManyRVDa develops *CachePool*, a scalable scalar–vector manycore architecture centered around a logically shared but physically distributed cache hierarchy. The design seeks to retain a simple shared-memory programming model while exposing sufficient control over data placement, memory sharing, synchronization, and execution resources to adapt the architecture to different workload characteristics.

The main design principles of CachePool are summarized below.

1. *Tightly coupled scalar–vector execution with configurable resource sharing.*

CachePool combines lightweight Snitch scalar cores with compact Spatz vector coprocessors. Scalar cores handle control-intensive and irregular execution, while Spatz accelerates data-parallel regions using multiple vector memory ports and SIMD arithmetic lanes. The tight accelerator interface allows scalar and vector execution phases to alternate at fine granularity without requiring a separate offload runtime.

CachePool additionally supports both 1S1V and 2S1V Core Complexes. In the latter configuration, two Snitch cores share one Spatz instance, allowing the scalar-to-vector resource ratio to be adapted to workload requirements while reducing the area cost of replicated vector units.

2. *A logically shared, physically distributed L1 cache.*

The L1 data cache is exposed as one shared memory space while its banks are physically distributed across Tiles and Groups. This organization avoids replication of independently cached copies and therefore eliminates the need for a conventional hardware cache-coherence protocol in the default configuration.

The physically distributed organization allows aggregate capacity and bandwidth to scale with system size, while the hierarchical interconnect provides access to both local and remote banks. The resulting architecture therefore combines a shared-memory abstraction with a physically non-uniform access topology.

3. *Software-controlled data placement and cache partitioning.*

Because physical access distance matters even within a logically shared cache, CachePool exposes programmable address mapping and partitioning mechanisms. Software-controlled address scrambling adjusts the interleaving granularity used to distribute cache lines across banks, Tiles, and Groups. This allows highly parallel kernels to spread traffic for bandwidth, while locality-sensitive kernels can keep larger contiguous regions closer to their consumers.

The same interconnect also implements configurable private/shared cache partitions independently of the cache controller. Any subset of the four banks in a Tile can be assigned to a private region, with private scope selected at either Tile or Group granularity, while the remaining banks stay globally shared. The fully private configuration can consequently decompose the machine into smaller logical clusters when stronger locality or isolation is beneficial.

4. *Scalable non-blocking cache control.*

CachePool integrates the In-Situ Cache controller as the shared L1 cache controller. Rather than relying on dedicated miss-status and write-buffer storage that grows separately from the cache capacity, In-Situ Cache repurposes cache-line resources to track outstanding read and write misses. This allows miss-handling capacity to scale with available cache resources while maintaining a compact hardware organization.

The controller is complemented by request coalescing, single-way lookup, physical-design-oriented replacement and metadata optimizations, and a wide cache-line refill interface. Together, these features target the large number of concurrent independent misses generated by sparse and irregular dataplane workloads.

5. *Hierarchical interconnects matched to access granularity.*

CachePool separates fine-grained core-generated memory traffic from cache-line refill traffic and uses different interconnect organizations for the two.

Within a Tile, multiple parallel all-to-all crossbars connect corresponding scalar and vector memory ports from the Core Complexes to the local L1 banks. Decomposing a monolithic crossbar by memory-port class reduces routing complexity while preserving parallel bandwidth.

Within and across Groups, fine-grained load/store traffic is carried through hierarchical local forwarding and multiple independent narrow meshes. Several parallel 32 bit mesh paths can be assigned to each Tile position, allowing bandwidth and routing resources to be adapted to physical-design constraints.

Cache-line refills use a separate 512 bit two-dimensional mesh connected to distributed off-chip memory endpoints. Separating the narrow core-access network from the wide refill network avoids forcing both traffic classes onto a single interconnect optimized for neither.

6. *Hierarchical synchronization and lightweight runtime control.*

CachePool provides hardware support for synchronization at the Tile, Group, and system levels. The hierarchical barrier supports partial participation as well as multiple independent barrier slots, enabling synchronization domains to match the physical and logical decomposition of the machine.

Memory-mapped configuration registers expose cache partitioning, address mapping, cache maintenance, barrier participation, Spatz ownership, and other architectural controls to software. A lightweight runtime abstracts these interfaces and provides topology discovery, synchronization, memory allocation, cache-management, and work-distribution primitives.

Based on these principles, the current ManyRVData implementation makes the following main contributions.

1. **A scalable multi-Group CachePool architecture.**

We developed and integrated a hierarchical manycore architecture composed of Groups, Tiles, and configurable scalar–vector Core Complexes. The current large-scale configuration supports a 4×4 Group mesh with four Tiles per Group and four Core Complexes per Tile, providing up to 256 scalar cores in the 1S1V organization and supporting larger hart counts with the 2S1V organization.

2. **Configurable scalar–vector Core Complexes.**

We integrated Snitch and Spatz into configurable 1S1V and 2S1V Core Complexes. The dual-scalar organization includes hardware support for safe Spatz sharing, including free and software-controlled lock modes and explicit ownership transitions.

3. **Distributed and partitionable shared-L1 organization.**

We implemented a 16 MiB logically shared L1 cache physically distributed across the full system. The L1 interconnect supports programmable address scrambling and mixed private/shared bank configurations, including Tile-private and Group-private regions. These mechanisms are implemented in the address-routing layer and remain independent of the cache-controller microarchitecture.

4. **In-Situ non-blocking L1 cache controller.**

We integrated and extended the In-Situ Cache architecture for CachePool. The controller reuses cache-line storage for unified read- and write-miss handling, supports dynamic expansion of miss metadata, and incorporates physical-design-oriented optimizations such as single-way lookup and simplified replacement. This provides scalable miss concurrency without requiring a dedicated MSHR structure sized independently of the cache.

5. **Hierarchical load/store and refill interconnects.**

We developed a multi-level interconnect architecture consisting of decomposed Tile-local crossbars, Group-local communication, multiple parallel narrow inter-Group meshes for fine-grained accesses, and a dedicated 512 bit refill mesh for cache-line traffic. The architecture is parameterized to adapt the number of routing resources and external-memory endpoints to different system scales and physical-design constraints.

6. **Hierarchical synchronization and system control.**

We implemented a hierarchical hardware barrier supporting Tile-, Group-, and system-level synchronization, partial participation, and multiple independent barrier slots. We also introduced a memory-mapped control interface for cache configuration, maintenance, address mapping, Spatz sharing, boot control, and execution status.

7. **Runtime and programming support.**

We developed a lightweight runtime that exposes the CachePool topology and architectural control mechanisms to software. The runtime provides core/Tile/Group identification, hardware and software barriers, atomic operations, memory fences, dynamic memory allocation, cache-maintenance operations, address-scrambling and cache-partition configuration, Spatz ownership control, and topology-aware work-distribution support.

8. **Fast system-level simulation with GVSoC.**

We developed a configurable GVSoC model of CachePool for fast end-to-end simulation and architectural exploration. The model executes the same application binaries as the RTL implementation and exposes major topology parameters at simulation time, allowing larger configuration sweeps without RTL re-elaboration. It includes models of the distributed cache and interconnect hierarchy and can optionally be coupled to DRAMSys for detailed off-chip memory timing.

9. **Dataplane-oriented workload and kernel development.**

We developed and adapted representative workloads including RLC dataplane kernels, linked-list and synchronization micro-kernels, dense and sparse linear-algebra kernels, and vectorized compute kernels. These workloads are used to characterize memory access behavior, guide architectural development, and evaluate the interaction between scalar control, vector execution, cache locality, synchronization, and memory-system bandwidth.

10. **Architectural side explorations.**

In parallel with the main shared-L1 architecture, we investigated several alternative design points, including private write-through L0.5 caches with both hardware-managed and software-managed coherence, folded-cache organizations, and alternative cache/interconnect organizations. These studies help quantify the trade-offs between locality, coherence complexity, miss latency, interconnect traffic, and implementation cost.

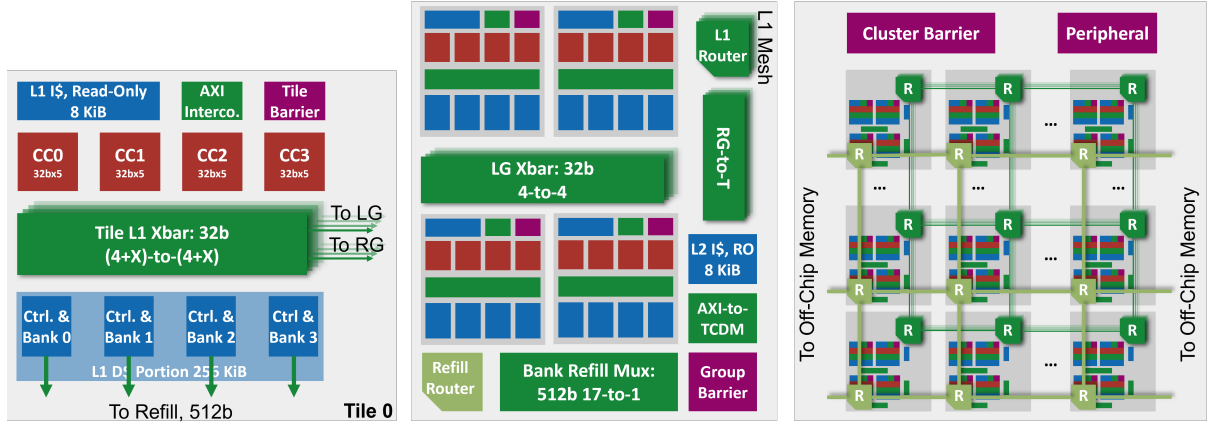


Figure 1: Top-level organization of CachePool, showing the hierarchical arrangement of Groups, Tiles, and Core Complexes together with the distributed shared L1 memory system and the main interconnect structure.

3 CachePool Architecture

CachePool is the manycore architecture developed within the ManyRVDData project. It combines heterogeneous scalar–vector processing elements with a physically distributed, software-configurable shared L1 data cache. The current system is organized hierarchically into 16 Groups connected through a 2D mesh. Each Group contains 4 Tiles, and each Tile integrates 4 Core-Complexes (CCs). This hierarchy provides a scalable organization for both the compute resources and the memory interconnect, while constraining long-distance communication to maintain physical implementation feasibility.

The memory system distinguishes between fine-grained core-generated memory accesses and cache-line refill traffic. Load and store requests are transported through a hierarchy of local crossbars and multiple narrow 2D meshes, whereas cache-line refills are carried by a separate 512 bit-wide 2D mesh connected to a configurable number of HBM endpoints. The L1 data cache is physically distributed across all Tiles but forms a single logical memory in the default configuration. Software can further partition the cache at bank granularity into globally shared and Tile- or Group-private regions, allowing the effective memory-sharing domain to be adapted to the workload.

3.1 System Overview

The current CachePool configuration comprises 16 Groups, with 4 Tiles per Group and 4 CCs per Tile. Its physical compute hierarchy can therefore be summarized as

$$\text{System} \rightarrow 16 \times \text{Group} \rightarrow 4 \times \text{Tile} \rightarrow 4 \times \text{CC}.$$

Each CC combines one or more scalar Snitch cores with a Spatz vector coprocessor, as detailed in Section 3.2.

Each Tile contains four L1 data-cache banks with a combined capacity of 256 KiB. Across the 64 Tiles in the system, these banks form a physically distributed 16 MiB L1 data cache. In the default configuration, all banks belong to a single globally shared address space and are accessible by every core in the system. Because the L1 is shared rather than replicated in private coherent copies, CachePool does not require a hardware cache-coherence protocol between cores.

The terms *Group* and *Tile* refer to fixed levels of the physical implementation hierarchy. In contrast, throughout this report, we use *cluster* to denote a logical set of cores sharing one L1 memory domain. This distinction becomes relevant when cache partitioning is enabled.

Software can restrict selected L1 banks to a Tile or Group, thereby changing the effective cluster organization without modifying the underlying physical Group–Tile topology.

3.2 Core Complex

3.2.1 Snitch Scalar Core

ManyRVData employs single-stage 32 bit RISC-V *Snitch* cores [39] as scalar host processors, supporting the RV32I base Instruction Set Architecture (ISA). Vector instructions are offloaded to the Spatz vector coprocessor, while optional 32 bit floating-point support can be enabled depending on the selected system configuration.

As in MemPool and TeraPool, the Snitch Load Store Unit (LSU) integrates a scoreboard to track outstanding memory operations and to guarantee forward progress in the absence of read-after-write (RAW) hazards. The LSU supports multiple outstanding requests, allowing Snitch to issue a sequence of loads and stores without waiting for earlier responses to return [40]. This capability is particularly important in CachePool, where accesses may target local banks, remote Tiles, remote Groups, or off-chip memory through the cache hierarchy. As a result, memory responses may return with non-uniform latency, while the core still preserves the required architectural ordering.

3.2.2 Spatz Vector Coprocessor

The streamlined 32 bit vector coprocessor *Spatz* [41] is tightly coupled to one or two Snitch scalar cores within a CC. In the current ManyRVData configuration, each Spatz instance contains 4 vector lanes and exposes four independent 32 bit memory ports, providing an aggregate load/store width of 128 bit per cycle before interconnect contention.

Spatz implements RVV 1.0 through a generic accelerator interface and uses a compact latch-based Vector Register File (VRF). Its controller employs a scoreboard to track dependencies and to support vector chaining across instructions. The dedicated Vector Load Store Unit (VLSU) can issue multiple outstanding memory requests, enabling the vector unit to tolerate the variable access latency introduced by the distributed L1 and the remote memory hierarchy. Optional 32 bit floating-point support can be enabled for both scalar and vector floating-point operations.

3.2.3 Core-Complex Configurations

ManyRVData supports two CC organizations:

- **1 Snitch + 1 Spatz (1S1V):** one scalar core is tightly coupled to one four-lane Spatz coprocessor.
- **2 Snitch + 1 shared Spatz (2S1V):** two scalar cores share one four-lane Spatz coprocessor, increasing scalar-thread density while amortizing the vector resources.

The selected organization is a compile-time system parameter. It also determines the number of independent narrow memory ports that must be routed from each CC to the local L1 banks.

For the 2S1V configuration, an Finite State Machine (FSM) controls the sharing of the Spatz interface between the two Snitch cores. Two operating modes are provided for different execution scenarios:

- **Free Mode:** requests from the two Snitch cores are forwarded to Spatz in a round-robin manner. Because Spatz can retire instructions out of order internally, the sender can only be switched after all previously accepted instructions from the current sender have completed. This is the default mode after reset.

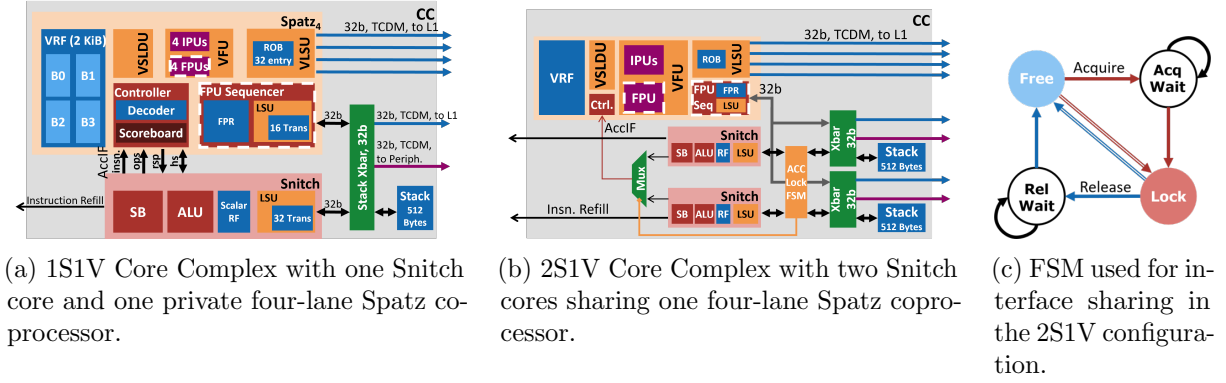


Figure 2: Core-Complex organizations supported in CachePool and the control mechanism used for vector-resource sharing. Panels (a) and (b) show the two supported CC configurations, while panel (c) shows the FSM that manages access arbitration and ownership transitions when two Snitch cores share one Spatz coprocessor.

- **Lock Mode:** one core can claim ownership of Spatz in software. In this mode, Spatz accepts requests only from the owner core. Requests issued by the non-owner are rejected immediately and return an error response.

Transitions between modes may not complete immediately because instructions can still be in flight inside Spatz. Two intermediate transition states are therefore introduced to prevent race conditions during mode switching. The resulting FSM is shown in Figure 2c.

3.2.4 Core-Local Stack Memory

One stack implementation provides each Snitch core with a dedicated 1024 B SRAM for low-latency private stack accesses. All cores observe the same virtual address mapping for this SRAM, meaning that a stack address always resolves to the local SRAM associated with the issuing core rather than to a globally unique physical SRAM instance.

When the private stack capacity is insufficient, stack accesses can be redirected to the shared L1 cache. The backing region in L1 is computed as a function of the core identifier, so that each core is assigned a distinct overflow region. This organization preserves the convenience of a private stack abstraction while allowing deeper stack usage when required. The address mapping and overflow handling are transparent to the programmer.

3.3 Tile Architecture

3.3.1 Tile Organization

Each Tile integrates 4 CCs and 4 L1 data-cache banks with a combined capacity of 256 KiB. The Tile also contains the local levels of the instruction-cache hierarchy together with the interconnect resources required to access its local L1 banks and to forward requests toward neighboring Tiles.

In the 1S1V configuration, each CC contributes one Snitch memory port and four Spatz memory ports. In the 2S1V configuration, each CC contributes two Snitch memory ports and four Spatz memory ports. These memory ports remain logically separated throughout the Tile-local access network to preserve parallelism and avoid concentrating all traffic on a single shared datapath.

3.3.2 Local L1 Access Network

Local L1 accesses are routed through multiple parallel all-to-all crossbars connecting the CC memory ports to the four L1 banks within the Tile. Each logical memory-port position is

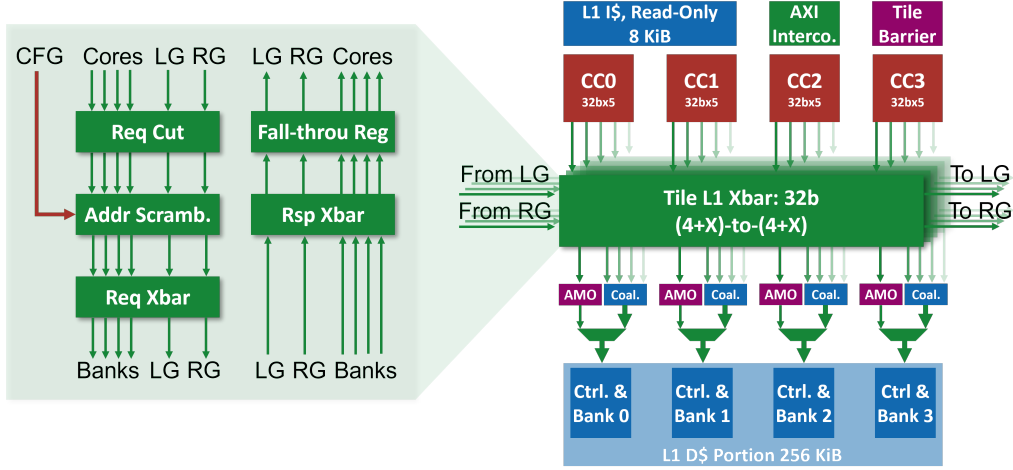


Figure 3: Tile-level organization of CachePool, showing the four Core Complexes, the local L1 data-cache banks, and the Tile-local interconnect resources.

assigned an independent crossbar that connects the corresponding port of every CC to the four local banks. Consequently, the Tile-local network contains 5 parallel crossbars in the 1S1V configuration and 6 parallel crossbars in the 2S1V configuration.

This decomposition avoids concentrating all requests into a single wide monolithic crossbar and provides independent arbitration for the Snitch and Spatz memory ports. For a crossbar with fixed data width, the switching complexity scales approximately as $\mathcal{O}(N_{in}N_{out})$. A flat organization with $N_{cc}N_{port}$ request inputs and $N_{port}N_{bank}$ output ports therefore scales as

$$\mathcal{O}_{flat} = \mathcal{O}(N_{cc}N_{port}^2N_{bank}).$$

In CachePool, the request ports are instead decomposed into N_{port} independent crossbars, each connecting N_{cc} inputs to N_{bank} local banks. The resulting aggregate complexity scales as

$$\mathcal{O}_{decomp} = \mathcal{O}(N_{port}N_{cc}N_{bank}),$$

removing one factor of N_{port} compared with the flat organization. Besides reducing switching and arbitration complexity, this decomposition improves physical routing feasibility by distributing the wiring across several narrower structures.

3.3.3 Atomic Operations

Although the globally shared L1 organization removes the need for a hardware cache-coherence protocol, concurrently executing cores still require synchronization and memory-consistency primitives. An atomic unit is therefore inserted on the scalar-core memory path before the L1 data-cache controller to provide Atomic Memory Operations (AMOs) in compliance with the RISC-V “A” extension. Vector requests bypass this unit because atomic operations are currently supported only on the scalar path.

The atomic unit was originally designed for SPM-based systems with predictable access latency. In CachePool, its control logic is adapted to use handshake-driven request and response handling so that it can tolerate the variable latency introduced by cache hits, cache misses, and remote accesses.

Spatz memory requests instead bypass the atomic unit and are forwarded directly to the request coalescer, described in Section 3.6.1, before entering the cache controller.

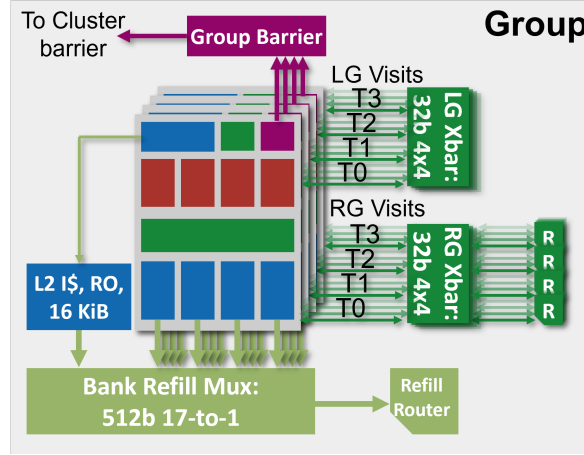


Figure 4: Group-level organization of CachePool, showing the four Tiles, the Group-local L1 interconnect, the inter-Group narrow network, and the wide refill NoC.

3.4 Group Architecture

Each Group contains 4 Tiles and forms the intermediate physical hierarchy between the Tile-local resources and the global inter-Group network. The default configuration contains 4 Tiles per Group, while the number of Tiles remains parameterizable for reduced system configurations and faster verification runs. The Group provides local communication resources that allow accesses to be forwarded between its constituent Tiles without entering the inter-Group mesh.

The four Tiles occupy fixed relative positions within the Group. These positions are preserved across the global 2D mesh and define the geometric correspondence used by the narrow inter-Group networks. A remote request can therefore reach the Tile at the same relative position directly through the corresponding mesh, while requests targeting another Tile position require an additional Group-local forwarding stage.

The Group also defines one of the supported visibility domains for private L1 cache partitions. When Group-private partitioning is enabled, banks assigned to the private region are accessible by all cores within the Group but remain inaccessible to cores in other Groups.

The Group additionally contains the shared L2 instruction-cache level, which serves the instruction caches of all Tiles in the Group before refill traffic is forwarded to the global 512 bit refill mesh.

3.5 Distributed and Partitionable L1 Data Cache

3.5.1 Global Shared-L1 Organization

The CachePool data cache forms a single logical L1 memory while being physically distributed across all Tiles in the system. Each Tile contains four L1 banks with a combined capacity of 256 KiB, resulting in a total capacity of 16 MiB across the 64 Tiles.

In the default configuration, all L1 banks are globally accessible by every core in the system. Because the architecture does not maintain mutually incoherent private copies of cached data, no hardware cache-coherence protocol is required between cores. Instead, all cores access the same logical L1 memory through the hierarchical load/store interconnect described in Section 3.7.

3.5.2 Address Mapping and Software-Controlled Scrambling

The distributed L1 banks are interleaved through a software-controlled address-scrambling mechanism. Software can configure the scrambling granularity according to the locality and bandwidth requirements of individual workloads. The scrambling granularity must be larger

than the 64 B cache-line size so that all bytes in one cache line map consistently to the same destination cache bank.

The programmable mapping provides a trade-off between locality and traffic distribution. A coarser scrambling granularity keeps larger contiguous address regions within the same Tile or Group and can therefore preserve spatial locality. A finer scrambling granularity distributes consecutive regions across more Tiles and banks, increasing the available aggregate memory bandwidth for highly parallel accesses. The scrambling parameters are programmed through memory-mapped system configuration registers, as described in Section 5.

3.5.3 Cache Partitioning

CachePool supports software-controlled partitioning of the four L1 banks in each Tile into *private* and *shared* regions. Partitioning is performed at the granularity of complete cache banks, resulting in the configurations summarized in Table 1.

Table 1: Supported private/shared L1 bank partitioning configurations within each Tile.

Private banks	Shared banks	Configuration
0	4	Fully shared (default)
1	3	Mixed
2	2	Mixed
3	1	Mixed
4	0	Fully private

Banks assigned to the shared region remain globally accessible by all cores in the system. For banks assigned to the private region, software additionally selects one of two visibility scopes:

- **Tile-private:** private banks are accessible only by cores located in the corresponding Tile.
- **Group-private:** private banks are accessible only by cores located in the corresponding Group.

The fully private configuration decomposes the system into multiple smaller logical clusters, where each cluster comprises the cores sharing the corresponding private L1 memory. Mixed configurations retain a globally accessible shared region for communication and shared data structures while providing private cache capacity and bandwidth to individual Tiles or Groups.

Cache partitioning is implemented through the address-mapping and scrambling logic in the L1 interconnect rather than within the cache controller itself. The mechanism is therefore independent of the particular cache-controller implementation used behind the L1 crossbar.

The partitioning scheme is controlled through memory-mapped configuration registers. One register selects the number of private and shared banks and the visibility scope of the private region. A second register defines the address boundary between the private and shared portions of the memory space. The address comparison determines whether an access is routed to the private or shared bank subset according to the configured partitioning scheme. The address-boundary setting is only active for mixed private/shared configurations and has no effect when all banks are configured as either fully shared or fully private.

3.6 L1 Cache Controller

3.6.1 Request Coalescer

A request coalescer is placed in front of the L1 cache controller to aggregate multiple narrow 32 bit memory requests targeting the same cache line into a wider request. This reduces redundant accesses to the cache controller and improves utilization of the 512 bit cache-line datapath.

The coalescer operates over a configurable request window. In the default configuration, the window contains two request slots, while the width can be adjusted at design time according to the expected access pattern and implementation constraints.

3.6.2 In-Situ Cache Overview

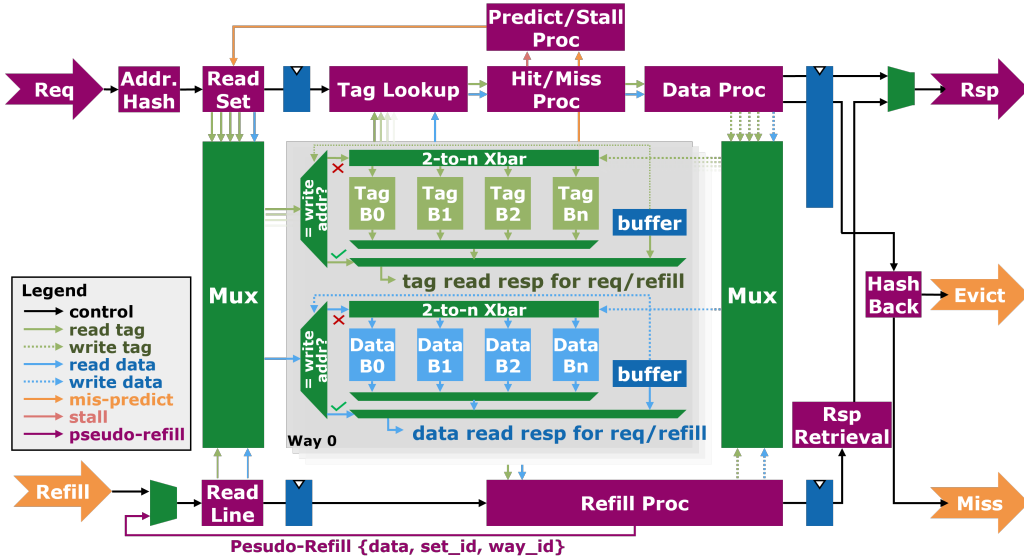


Figure 5: Block diagram of the In-Situ Cache controller, showing the request, tag/data access, refill, response, and eviction paths.

The L1 banks are managed by the *In-Situ Cache* controller [42], a scalable non-blocking cache architecture designed to tolerate long and variable refill latencies without relying on large dedicated miss-handling structures. Instead of storing outstanding-miss state in a conventional Miss Status Handling Register (MSHR) array and separate write buffer, In-Situ Cache repurposes otherwise available cache lines to track misses directly within the cache structure.

The controller extends the conventional invalid/valid cache-line state model with dedicated pending states for read and write misses. This enables unified handling of outstanding read misses, sub-cacheline write misses, and request merging through the same tag lookup mechanism. Because the miss-handling state is stored in cache-line resources, the available tracking capacity naturally scales with the cache itself rather than being fixed by a statically provisioned MSHR structure.

The current implementation further incorporates a set of microarchitectural and physical-design optimizations, including dynamic expansion of read-miss metadata, single-way data lookup through hash-based way prediction, pseudo-random replacement, and a folded cache organization. The folded-cache implementation and its physical-design implications are discussed separately in Section 3.6.6.

3.6.3 Unified In-Cacheline Miss Handling

In-Situ Cache extends the conventional invalid/valid cache-line state model with two additional states, *read-pend* and *write-pend*, allowing outstanding read and write misses to be represented directly within cache lines. A hit on a pending line indicates that a request can be merged with an already in-flight miss, while a miss on all valid and pending states allocates a replacement line for miss tracking.

For a read miss, the selected cache line transitions to the read-pend state and records the metadata required to return the refill data to the requesting processing element. Subsequent

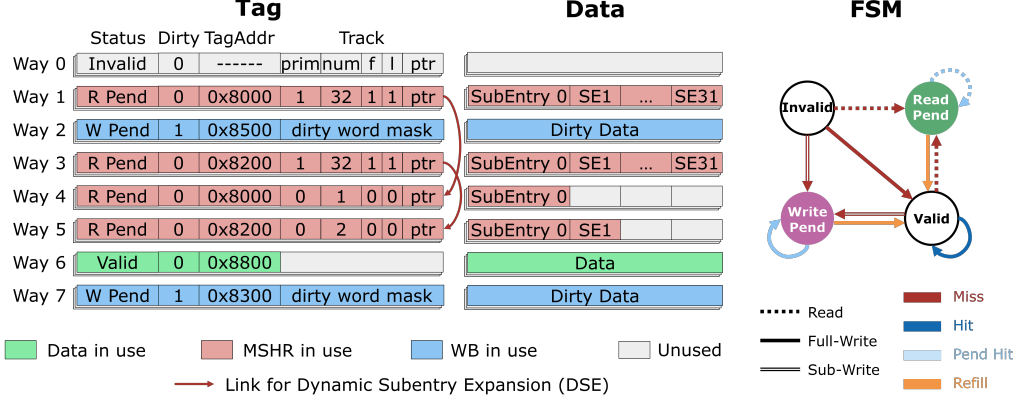


Figure 6: Cache-line organization and state transitions of the In-Situ Cache controller, including valid, read-pending, and write-pending states.

requests to the same cache line can merge into the existing pending entry rather than issuing redundant refill requests. When the refill returns, the line transitions to the valid state and responses are generated for all merged requesters.

Sub-cacheline write misses are handled similarly through the write-pend state. The cache line stores the partially updated data together with a dirty-word mask that records which words have been modified while the refill is outstanding. Once the missing cache block arrives, the pending writes are merged with the refill data and the line transitions to the valid state.

The design therefore unifies read-miss tracking and write-miss coalescing within the cache itself, eliminating the need for separate dedicated MSHR and write-buffer storage.

3.6.4 Request and Refill Pipelines

The In-Situ Cache controller is organized as a three-stage pipeline with separate control paths for processing-element requests and memory refills. The request path performs address hashing and way prediction in the first stage, tag comparison and prediction checking in the second stage, and hit response or miss generation in the final stage.

The refill path writes returned cache-line data directly to the location selected when the miss was allocated. For read-pending lines extended through DSE, the refill controller traverses the linked expansion lines and generates pseudo-refill operations to recover all request metadata and invalidate the expansion entries. This allows refill completion and response generation without requiring a separate miss-response buffer.

To avoid deadlock and guarantee forward progress, refill traffic is given bounded access to the internal structures and is only admitted when the required landing resources are available. Tag and data memories are implemented as pseudo-dual-port structures constructed from single-port SRAM macros, with forwarding and fixed-priority arbitration used to resolve same-bank conflicts.

3.6.5 Way Prediction and Replacement

A conventional set-associative cache reads all data ways on every lookup, which increases SRAM access energy and routing activity. In-Situ Cache instead uses a hash-based way-prediction mechanism that selects one data way for the initial access while all tag ways are still checked in parallel.

If the predicted way is incorrect, the request is replayed to access the actual hit way or another eligible replacement way. The controller combines this mechanism with pseudo-random replacement, avoiding the additional state and update logic required by an LRU policy while also excluding pending lines from replacement.

This organization reduces data-SRAM activity and simplifies replacement logic, while the occasional replay introduces only a small additional latency for mispredicted accesses.

3.6.6 Folded Cache Organization

TODO: Add details here.

3.7 Hierarchical Load/Store Interconnect

The routing decision is performed hierarchically based on each request’s physical destination. The interconnect first determines whether the target bank resides in the issuing Tile, elsewhere in the same Group, or in a remote Group. Only the network resources required for the selected access class are traversed, avoiding unnecessary propagation through higher-level interconnect structures.

3.7.1 Interconnect Organization and Protocol

Core-generated L1 traffic is carried by a hierarchical narrow interconnect organized into three access classes: intra-Tile, intra-Group, and inter-Group. Fine-grained load/store accesses use a lightweight TCDM-style request/response protocol rather than a full AXI fabric, reducing interface complexity, wiring overhead, and ordering requirements for narrow memory transactions. The request, response, and user-metadata fields of the interface are summarized in Table 2. The width of the `core_id` field depends on the selected CC configuration, since a Tile contains either four or eight Snitch scalar cores.

The local crossbars described in Section 3.3.2 handle accesses targeting banks within the issuing Tile. Requests targeting another Tile in the same Group are forwarded through Group-local crossbar resources. Accesses to remote Groups are transported through multiple narrow 2D meshes and are optionally followed by an additional Group-local forwarding stage when the destination Tile is not geometrically aligned with the incoming mesh.

3.7.2 Intra-Tile Access

An access targeting an L1 bank in the issuing Tile follows the direct path

CC memory port $\rightarrow$ local parallel Xbar $\rightarrow$ local L1 bank.

This is the shortest access path in the hierarchy and does not enter either the Group-level interconnect or the inter-Group meshes.

3.7.3 Intra-Group Access

For accesses targeting another Tile within the same Group, each Tile exposes a configurable number of ports toward the Group-local crossbar network. The default configuration provides two such ports per Tile.

The outgoing Group-local port is selected as a modulo function of the issuing core ID. This distributes requests from different cores across the available ports and increases the aggregate bandwidth available for accesses to neighboring Tiles. The number of Group-local ports and the corresponding source-to-port mapping are compile-time configurable to match the routing resources available in a particular physical implementation.

Table 2: TCDM-style request/response interface used between processing elements and the L1 memory system.

Request		
Field	Bits	Description
addr	32	Target byte address
write	1	Load/store selector
amo	4	Atomic-operation encoding
data	32	Store request data
strb	4	Byte-enable mask for stores
size	3	Transfer-size encoding preserved from AXI transactions
user	15/16	User-defined metadata
q_valid	1	Request-valid handshake
p_ready	1	Response-ready handshake
Response		
Field	Bits	Description
data	32	Load response data
error	1	Error indicator
write	1	Store-response indicator
user	15/16	User-defined metadata
p_valid	1	Response-valid handshake
q_ready	1	Request-ready handshake
User Metadata		
Field	Bits	Description
core_id	2/3	Identifier of the sending scalar core within a Tile
tile_id	6	Identifier of the sending Tile
is_amo	1	Marks responses associated with atomic operations
req_id	5	Request identifier used exclusively for response reordering
is_fpu	1	Indicates requests originating from the floating-point path

3.7.4 Inter-Group Narrow Meshes

Accesses targeting a remote Group are transported through multiple narrow 2D meshes with a topology similar to TeraNoC [38]. Each mesh hop introduces two cycles of forwarding latency due to the input and output buffering implemented at the routers. These pipeline stages are required to meet timing across the long inter-Group wires imposed by the physical dimensions of a Group.

Each Tile is associated with multiple 32 bit-wide mesh paths that connect geometrically corresponding Tile positions across neighboring Groups. In the default configuration, four parallel narrow meshes are instantiated per Tile position. This configuration has been verified to be routable in the physical design.

Using several narrow meshes in parallel increases aggregate inter-Group bandwidth without introducing a single physically wide global load/store fabric. The organization also preserves the native 32 bit granularity of scalar and vector memory-port requests throughout the long-distance interconnect.

3.7.5 Non-Geometric Destination Routing

The narrow meshes naturally terminate at the Tile occupying the same relative position in the destination Group. If a request targets a different Tile position within that Group, an additional local forwarding stage is required.

Such an access follows the path

source Tile $\rightarrow$ narrow mesh $\rightarrow$ destination-Group Xbar $\rightarrow$ destination Tile.

The inter-Group mesh performs the long-distance transport between Groups, while the Group-local crossbar resolves the final Tile destination. This two-stage organization avoids requiring every narrow mesh to connect directly to every Tile position in neighboring Groups, thereby reducing global wiring complexity.

3.7.6 L1 Network Latency

The latency of an L1 access depends on the physical location of the destination bank and the corresponding interconnect resources traversed by the request and response. We characterize the zero-load round-trip latency from a scalar core to the destination L1 data SRAM, including the return path to the issuing core. The request path starts at the core and terminates at the destination data SRAM, while the response path carries the returned data back to the requester through the corresponding return-routing resources. One cycle of the request and response paths overlaps at the SRAM access, such that the round-trip latency is one cycle shorter than the sum of the two independently counted paths.

Table 3: Zero-load L1 access latency for representative destination classes.

Destination	Request	Response	Round Trip
Same Tile	4	3	6
Same Group	5	5	9
Neighbor Group, geometric Tile	9	9	17
Neighbor Group, non-geometric Tile	10	10	19

An intra-Tile access requires 4 cycles on the request path and 3 cycles on the response path, resulting in a 6-cycle round-trip latency. The request traverses the core-side pipeline cut, scalar routing crossbar, local L1 crossbar, atomic unit, cache controller, and data SRAM, while the response returns through the corresponding local structures.

An access to another Tile within the same Group additionally traverses the Group-local crossbar network. This increases both the request and response paths to 5 cycles and results in a 9-cycle round-trip latency.

For an access to the geometrically corresponding Tile in a neighboring Group, the request enters the narrow 2D mesh directly from the source Tile and traverses the input and output pipeline stages of both the source and destination routers. The request and response paths each require 9 cycles, giving a total round-trip latency of 17 cycles.

If the destination is located in a different relative Tile position within the remote Group, an additional local NoC crossbar stage is required to select the appropriate narrow mesh and complete the final Tile routing. This adds one cycle to both the request and response paths, increasing the neighboring-Group round-trip latency from 17 to 19 cycles.

Each inter-Group router contains one pipeline cut at its input and one at its output. Consequently, every additional mesh hop beyond the neighboring Group adds 2 cycles to the request path and 2 cycles to the response path, corresponding to 4 additional round-trip cycles.

For an inter-Group access, the zero-load round-trip latency can therefore be expressed as

$$L_{RT} = 17 + 4(N_{\text{hop}} - 1) + 2I_{\text{nongeo}},$$

where $N_{\text{hop}} \geq 1$ is the number of inter-Group mesh hops and I_{nongeog} is 1 when an additional non-geometric Tile-routing stage is required and 0 otherwise. The first term represents access to the geometrically corresponding Tile in a neighboring Group, the second captures the additional router stages for longer-distance accesses, and the final term accounts for the extra crossbar traversal required by non-geometric destinations.

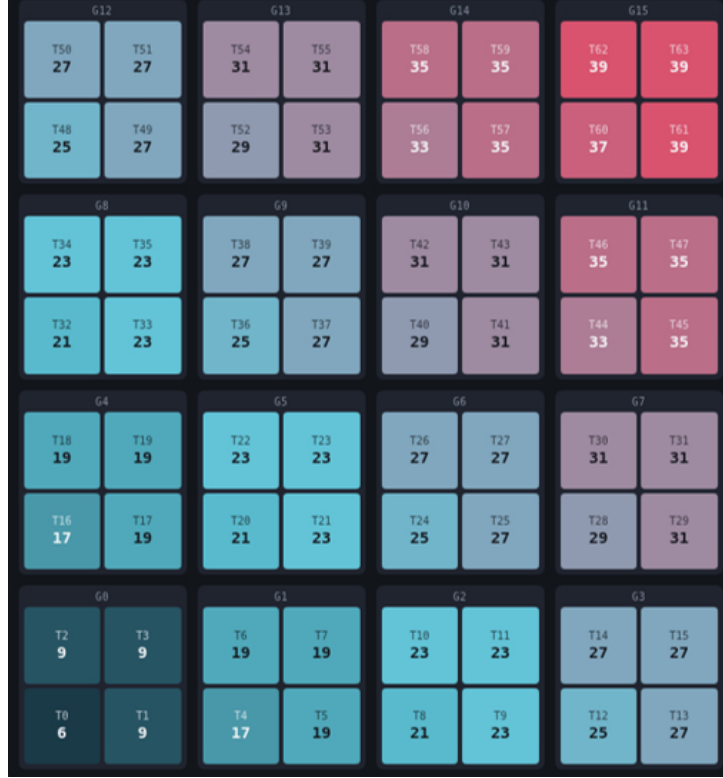


Figure 7: Zero-load round-trip L1 access latency from Core 0 to cache-bank destinations across the CachePool topology. The latency increases with the number of inter-Group mesh hops and includes an additional penalty when the target is not located at the geometrically corresponding Tile position.

Figure 7 illustrates the resulting spatial latency distribution across the system. Banks within the issuing Tile provide the minimum latency, while accesses become progressively more expensive as they traverse the Group-local network and additional inter-Group mesh hops. The non-uniform latency is therefore primarily determined by physical network distance rather than by a uniform logical shared-memory abstraction.

TODO: Add a reference to the appendix section describing the explored torus topology and its latency/1

3.8 Cache Refill and Main-Memory Network

3.8.1 Wide Refill Mesh

Cache-controller refill traffic is physically separated from the narrow core-generated load/store network. All cache controllers connect to a dedicated 512 bit-wide 2D mesh NoC that is used for full cache-line transfers between the distributed L1 cache and the external-memory interfaces. The width of the refill network matches the 64 B cache-line size, allowing an entire cache line to be transferred in one network beat when sufficient downstream bandwidth is available.

Separating refill traffic from fine-grained core accesses prevents wide cache-line transfers from consuming the routing resources of the narrow load/store interconnect. It also allows the two fabrics to be optimized independently for their different traffic characteristics: the narrow

network targets low-overhead word accesses, while the refill mesh is optimized for high-throughput cache-line movement.

3.8.2 HBM Connectivity

The wide refill mesh connects to a configurable number of HBM endpoints. The current CachePool configuration supports either 8 or 16 HBM channels, allowing the available off-chip bandwidth to be scaled according to the system configuration.

An example physical organization for the 16-channel configuration is shown in Figure 8. The HBM channels are distributed around the perimeter of the Group mesh, with neighboring channels placed on the same side of the system where possible. This arrangement is intended to match the physical packing constraints of typical multi-channel HBM controller IPs while keeping the average distance between Groups and external-memory endpoints short.

The placement also preserves a notion of affinity between Group identifiers and HBM channels. Channels are positioned such that Groups with matching or nearby identifiers can access a nearby HBM endpoint with a short path through the refill mesh. This organization aligns with the software convention used by bandwidth-oriented kernels, where off-chip traffic can be distributed across Groups and their preferred memory channels to reduce contention and improve aggregate bandwidth.

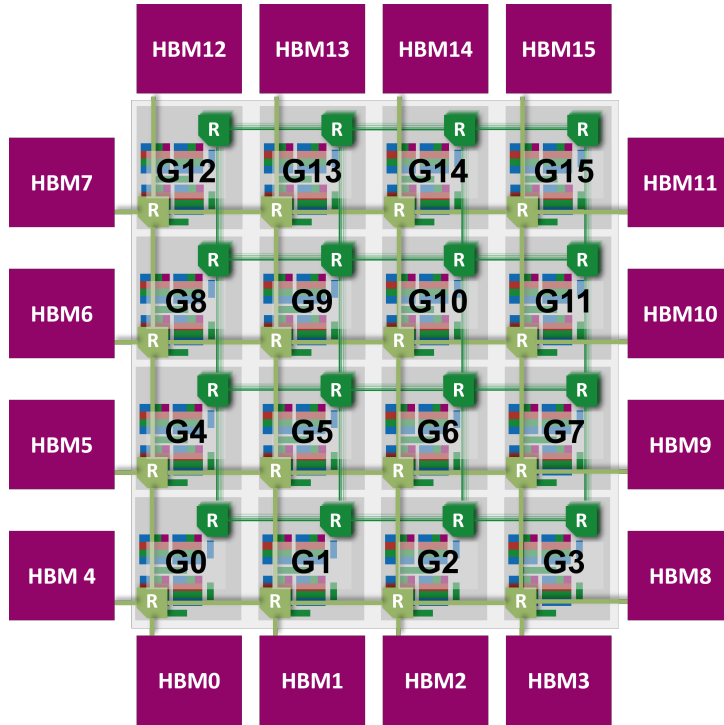


Figure 8: Example organization of the 16-channel HBM configuration and the wide refill mesh. The HBM endpoints are distributed around the perimeter of the 16-Group array to reduce communication distance and to provide a natural affinity between Groups and external-memory channels.

3.8.3 Refill Routing and Protocol Conversion

As described above, cache refill traffic is transported through a dedicated 512 bit-wide TCDM-style 2D mesh NoC. The refill network carries full cache-line requests and responses between the distributed cache controllers and the off-chip memory interfaces. Table 4 summarizes the fields used by the refill protocol.

Compared with the narrow core-access interface, several protocol fields are simplified because the refill datapath width matches the 512 bit cache-line width. For HBM-backed configurations, one cache line is transferred in a single network beat and no multi-beat burst reconstruction is required inside the refill mesh. The burst metadata is therefore reduced to the information required by the endpoints, while unused burst-length bits are tied to constants and removed during physical implementation.

The write-strobe field is likewise simplified. Refill transactions operate on complete cache lines, so the 64 bit strobe vector is driven either entirely high or entirely low rather than carrying independent per-byte enables. The resulting constant structure can be collapsed during synthesis and physical implementation.

Atomic-operation information is not required on the refill network. The `amo` field is therefore tied to `AMONONE` and is not consumed by any refill endpoint.

The refill request metadata identifies the originating cache bank and Tile and carries the information required to route returning data back to the corresponding cache-controller transaction. The protocol also preserves a source identifier used by the next memory-hierarchy level and cache-internal miss-handling metadata required when the refill response returns.

Table 4: TCDM-style request/response interface used by the cache-refill network.

Request		
Field	Bits	Description
<code>addr</code>	32	Target byte address
<code>write</code>	1	Read/write selector
<code>amo</code>	Z	Atomic-operation field, tied to <code>AMONONE</code>
<code>data</code>	512	Cache-line write data
<code>strb</code>	64	Byte-enable vector, driven uniformly to all zeros or all ones
<code>size</code>	3	Transfer-size encoding preserved from the external-memory transaction
<code>user</code>	26/28	User-defined refill metadata
<code>q_valid</code>	1	Request-valid handshake
<code>p_ready</code>	1	Response-ready handshake
Response		
Field	Bits	Description
<code>data</code>	512	Cache-line refill data
<code>error</code>	1	Error indicator
<code>write</code>	1	Write-response indicator
<code>user</code>	26/28	User-defined refill metadata
<code>p_valid</code>	1	Response-valid handshake
<code>q_ready</code>	1	Request-ready handshake
User Metadata		
Field	Bits	Description
<code>bank_id</code>	3	Identifier of the originating cache bank within a Tile
<code>tile_id</code>	6	Identifier of the originating Tile
<code>l2_src_id</code>	5	Source identifier used by the next memory-hierarchy level
<code>info</code>	11	Cache miss-handling metadata associated with the refill transaction
<code>burst</code>	1/3	Burst metadata; reduced when the cache-line width matches the data width

When the refill network width matches the cache-line width, only a single burst-control bit is required to distinguish data-cache and instruction-cache traffic. The remaining burst-length bits

are tied to constant values and are not consumed by the endpoints. In configurations requiring multi-beat refills, the full burst metadata can instead be retained.

3.9 Instruction Memory Hierarchy

ManyRVData employs a hierarchical instruction-cache organization that we refer to as a 2.5-level hierarchy:

per-core L0 iCache → Tile-shared L1 iCache → Group-shared L2 iCache → 512-bit refill mesh.

3.9.1 Per-Core L0 Instruction Cache

Each scalar core contains a small private L0 instruction cache of 128 B for the hottest instruction working set and lowest-latency instruction delivery.

3.9.2 Tile-Shared L1 Instruction Cache

The next level is shared by the cores within one Tile, reducing redundant instruction storage while keeping most instruction traffic local to the Tile. The L1 instruction cache has a default capacity of 8 KiB.

3.9.3 Group-Shared L2 Instruction Cache

A further instruction-cache level is shared across the Tiles of a Group. This level absorbs misses from the Tile-shared L1 instruction caches before they reach the refill network. The Group-shared L2 instruction cache has a default capacity of 16 KiB. Misses from this level enter the same 512 bit refill mesh used by the data-cache controllers.

3.10 System Control and Synchronization

CachePool provides lightweight hardware mechanisms for runtime configuration and synchronization across the manycore system. Memory-mapped control registers expose architectural features such as cache partitioning and address scrambling to software, while a hardware barrier provides scalable synchronization across cores and higher-level hierarchy domains.

3.10.1 Memory-Mapped Control and Status Registers

CachePool exposes a set of memory-mapped control and status registers for runtime configuration of system-level architectural features. The register interface covers synchronization, Spatz sharing, instruction-cache control, boot and execution status, L1 cache maintenance, cache partitioning, and address-mapping configuration. Most architectural mechanisms described in the previous subsections are therefore programmable without requiring a hardware rebuild.

The number of Tile-selection registers and hardware-barrier participation masks is parameterized through `NumTileSelRegs` and `NumBarrierSlots`, respectively. The default Tile-selection configuration provides two 32 bit masks, which together cover the 64 Tiles of the 16-Group system. The number of barrier slots follows the selected system configuration, as described in Section 3.10.2.

Several of the configuration registers use an explicit programming and commit sequence. This allows software to prepare a new configuration before atomically applying it to the corresponding hardware mechanism. Examples include the L1 SPM configuration, L1 maintenance operations, and crossbar address-mapping offset.

The software abstractions and runtime APIs used to program these registers are described in Section 5.

3.10.2 Hardware Barrier

CachePool provides a dedicated hierarchical hardware barrier to support low-overhead synchronization across the large number of scalar cores in the system. The barrier follows the physical organization of the architecture and is implemented as a three-level tree spanning Tile, Group, and system-wide domains. This hierarchical organization allows synchronization state to be aggregated locally before propagating to the next level, avoiding a flat global structure with large fan-in and long timing paths.

At the Tile level, participating Snitch cores report their arrival to a local barrier instance. The Tile-level state is then aggregated by the corresponding Group-level barrier, which in turn communicates with the top-level barrier for synchronization spanning the complete system. Pipeline cuts are inserted along the hierarchy to meet timing across the longer communication paths between hierarchy levels.

The barrier supports both *full* and *partial* synchronization. For a full barrier, all configured participants propagate through the complete Tile–Group–system hierarchy and are released only after the global synchronization condition has been satisfied. For a partial barrier, software can configure a participation mask such that synchronization terminates at a lower hierarchy level when all required local participants have arrived. A local partial barrier therefore does not need to propagate to higher levels of the hierarchy, reducing synchronization latency and avoiding unnecessary global traffic.

Multiple independent barrier instances are supported concurrently through a set of barrier slots. Each barrier operation is identified by a barrier ID, allowing different groups of threads or execution phases to maintain independent synchronization state. The default configuration provides 2 barrier slots for 1S1V systems and 4 slots for dual-scalar 2S1V systems, while the number of slots remains configurable at design time.

Barrier participation and partial-barrier masks are controlled through memory-mapped peripheral registers. The barrier state maintains the masks and identifiers required to track participating cores together with valid and completion information at the corresponding hierarchy levels. This provides software with a configurable synchronization primitive while keeping the latency-critical arrival and release operations in hardware.

The software interface and runtime support for configuring and invoking the hardware barrier are described in Section 5.

3.11 System Configuration and Scalability

CachePool is intentionally parameterized rather than fixed to a single microarchitectural configuration. The main architectural parameters include the number and arrangement of Groups, the number of Tiles and CCs per Group, the CC organization, optional 32 bit floating-point support, L1 cache geometry and controller options, local and remote interconnect resources, refill-network organization, and the number of external-memory channels. Several of these parameters are selected explicitly in the build configuration, while others are derived automatically from higher-level architectural choices.

The current RTL repository provides a set of predefined configurations covering different combinations of system scale, CC organization, floating-point support, and external-memory connectivity. For compactness, the common `cachepool_` prefix is omitted from the configuration names in Table 5.

3.12 DRAM/HBM Co-Simulation

A time-efficient yet cycle-accurate external-memory subsystem is required to evaluate data-transfer performance together with the RTL memory hierarchy. The simulation environment integrates *DRAMSys5.0* [43] through an RTL interface, enabling cycle-accurate modeling of

multiple external-memory technologies. The current CachePool environment supports both DDR4-2400 and HBM2E configurations, which can be selected through the system configuration files to evaluate the corresponding memory-bandwidth and latency characteristics.

Table 5: Predefined CachePool build configurations provided by the current ManyRVData RTL repository. The common `cachepool_` prefix is omitted from the configuration names.

Parameter	4g	fpu_4g	dual_4g	dual_fpu_4g	fpu_16g	fpu_16g_tiny	dual_fpu_16g
Groups	4	4	4	4	16	16	16
Topology	2×2	2×2	2×2	2×2	4×4	4×4	4×4
CC organization	1S1V	1S1V	2S1V	2S1V	1S1V	1S1V	2S1V
Spatz FP	No	Yes	No	Yes	Yes	Yes	Yes
HBM channels	4	4	4	4	16	8	16
Variant	Small-size	FP	Dual scalar	Dual scalar + FP	Baseline	Tiny	Dual scalar

Table 6: Summary of the memory-mapped control and status registers exposed by CachePool.

Category	Register	Function
Barrier and synchronization	HW_BARRIER	Load-based hardware barrier synchronization
	HW_BARRIER_PARTICIPATION_MASK[n]	Per-slot Group participation mask for system-wide barriers
Spatz ownership control	SPATZ_LOCK_ACQUIRE	Acquire Spatz ownership in the 2S1V configuration
	SPATZ_LOCK_RELEASE	Release Spatz ownership in the 2S1V configuration
Instruction-cache control	ICACHE_PREFETCH_ENABLE	Enable or disable instruction-cache prefetching
Execution status and profiling	SPATZ_STATUS	Indicate whether the system is executing a kernel
	SPATZ_CYCLE	Store software-visible kernel cycle counts
Boot and termination	CLUSTER_BOOT_CONTROL	Configure the post-boot program entry point
	CLUSTER_EOC_EXIT	Report end-of-computation and program exit status
L1 SPM/cache configuration	CFG_L1D_SPM	Configure the size of the L1 SPM region
	L1D_SPM_COMMIT	Commit the programmed SPM configuration
L1 maintenance	CFG_L1D_INSN	Select L1 flush or invalidation operations
	CFG_L1D_TILE_SEL[n]	Select Tiles targeted by private-partition maintenance operations
	L1D_INSN_COMMIT	Commit the selected L1 maintenance operation
	L1D_FLUSH_STATUS	Report whether an L1 flush or invalidation is in progress
L1 partitioning	L1D_PRIVATE	Configure the number of private banks and Tile/Group-private scope
	L1D_ADDR	Configure the address boundary of the private L1 region
L1 address mapping	XBAR_OFFSET	Configure the L1 crossbar address-mapping offset
	XBAR_OFFSET_COMMIT	Commit the programmed crossbar offset

4 Physical Implementation

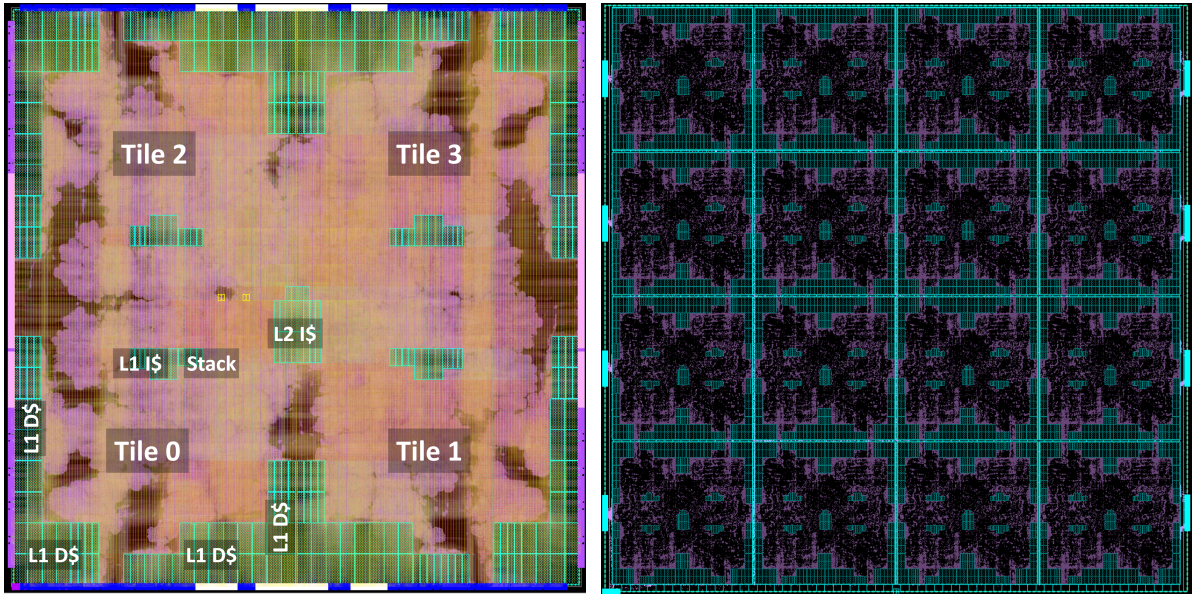
This section evaluates the physical feasibility and area efficiency of different CachePool single-tile cluster configurations, with particular focus on the impact of varying cacheline widths.

4.1 Methodology

We implement the CachePool cluster in TSMC’s 7 nm technology using a bottom-up approach. We first synthesize, place, and route the group level, then use it as a black box to assemble the cluster. All synthesis, placement, and routing are carried out using Synopsys Fusion Compiler 2025.06 flow. The current design targets 1 GHz at the worst-worst corner under m40C/125C at 0.675 V for the 7nm process. The configuration is set to the cachepool_fpu.16g for the baseline of physical design, targeting a 50-60% area utilization.

4.2 Floorplan

The floorplan size for the baseline design is $1670 \times 1690 \mu m$, yielding a 54% area utilization after routing. 16 groups are then assembled together in a cluster design, forming a $6900 \times 6920 \mu m$ cluster. A small gap of $30 \mu m$ is reserved for routing between groups. Slightly wider channels are reserved at the edge of the cluster for cluster-level logic.



Baseline configuration of a CachePool group. Baseline configuration of a CachePool cluster.

Figure 9: Placed-and-routed layouts of the CachePool group and cluster level of baseline configuration. **TODO: change to latest version**

4.3 Area Breakdown

The Gate Equivalent (GE), defined as the area of a two-input NAND gate, is a widely used metric to represent logic area in physical design independently of the technology node. The main area breakdown is derived from post-synthesis data, since the floorplan used in Figure 10 is not fully optimized. We also report a breakdown from post-layout results in Figure 11, but these numbers are more sensitive to floorplan variations and should be interpreted with caution.

The dominant contributor to the total area is the L1 memory subsystem, which accounts for roughly 60% of the total in the 128 b cacheline configuration with Floating Point Unit (FPU)

support. Increasing the cacheline size leads to a significant growth in both the data banks and the controller. The data bank overhead arises from the lack of native 128×512 SRAM macros, which forces us to emulate them using four 128×128 SRAMs. This results in poor area efficiency. Two possible remedies are under consideration:

- *Multi-muxed SRAMs*: A 128×512 configuration can be realized by multiplexing narrower SRAM macros. While still suboptimal, this approach improves efficiency compared to the current solution.
- *Bandwidth-matched SRAMs*: The effective core-to-cache bandwidth is only 128 b. Wider cachelines are mainly used for pseudo prefetching and bandwidth matching with DRAM. Therefore, using a 512×128 SRAM macro could achieve full core utilization without performance loss, while improving area efficiency.

The PEs together occupy about 30% of the total area, with approximately 900 kGE attributable to the FPUs.

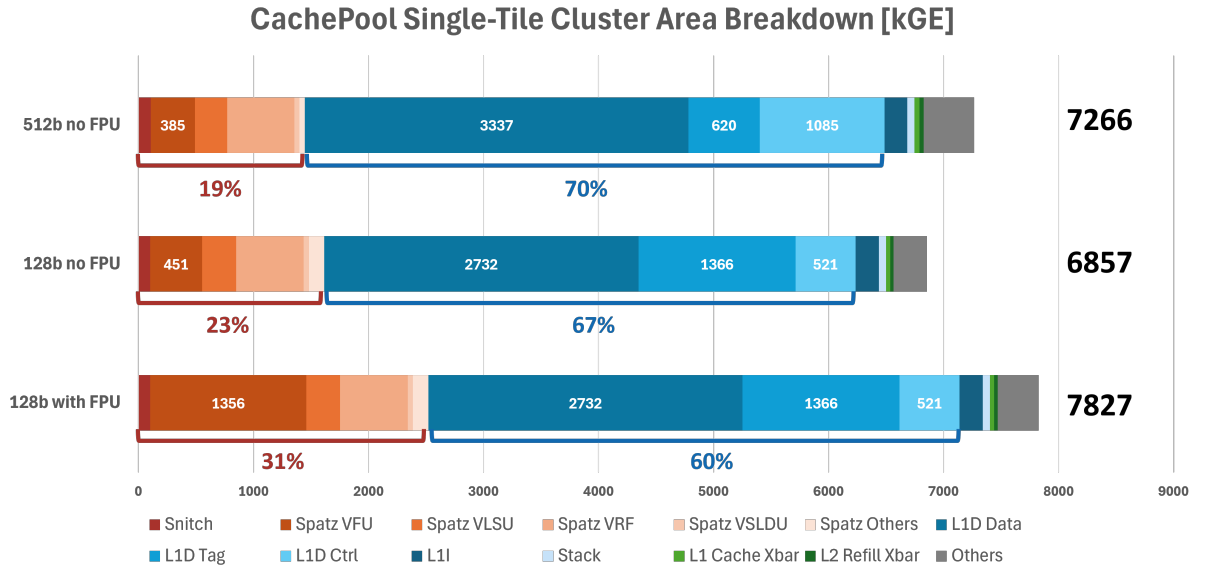


Figure 10: Post-synthesis area breakdown of the CachePool single-tile cluster. The 128 b configuration without FPU is estimated by subtracting the FPU area.

Overall, the total cluster area ranges from roughly 6.8 MGE (128 b cacheline without FPU) to 7.8 MGE (128 b cacheline with FPU), with the 512 b cacheline variant occupying about 7.3 MGE. This confirms that memory dominates the area budget, while FPUs contribute a significant but optional overhead. Interconnection logic and miscellaneous components remain below 5%, indicating they are not a limiting factor at this scale. Looking ahead, scaling CachePool to many tiles will be primarily constrained by SRAM efficiency rather than logic complexity, making optimized memory macros a key requirement for future implementations.

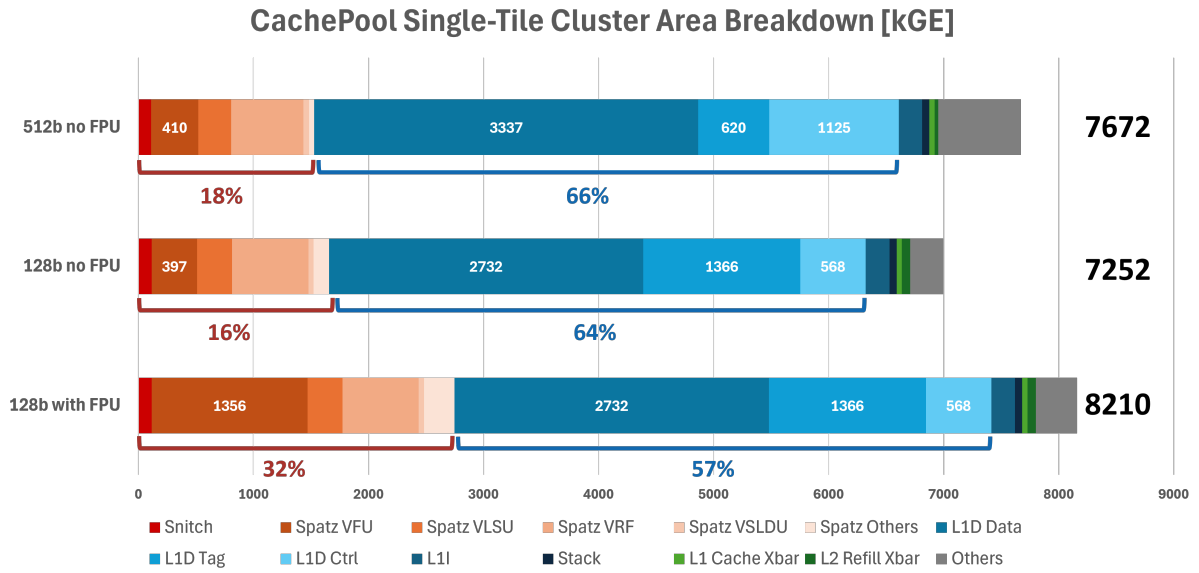


Figure 11: Post-route area breakdown of the CachePool single-tile cluster. The 128 b configuration without FPU is estimated by subtracting the FPU area.

5 Software and Runtime Support

The CachePool runtime provides the software interface required to expose the architectural mechanisms described in Section 3. It abstracts system topology, memory placement, cache configuration, synchronization, and execution control through a set of lightweight runtime functions and memory-mapped configuration interfaces. The runtime is designed to preserve the hierarchical organization of CachePool, allowing software to reason explicitly about cores, Tiles, Groups, and memory-sharing domains when required.

5.1 Programming Model

CachePool follows a fork-join programming model in which all Snitch cores execute a common program and synchronize at explicit barriers between parallel phases. A designated core performs initialization and other sequential tasks when required, after which work is distributed across the participating cores for parallel execution. The runtime provides topology and synchronization primitives that allow applications to organize work at the core, Tile, Group, and system levels.

The scalar Snitch processors are programmed primarily in C and compiled using the LLVM RISC-V toolchain. Scalar code handles control flow, work distribution, synchronization, memory management, and orchestration of vector computation.

Spatz implements the RISC-V Vector Extension (RVV) 1.0 programming model but currently supports only a subset of the complete RVV instruction set. In particular, Spatz does not implement several vector masking, fixed-point, and permutation operations. Consequently, unrestricted compiler auto-vectorization cannot be assumed to generate instruction sequences that are directly executable by Spatz.

The current ManyRVData software flow therefore disables or avoids unrestricted LLVM auto-vectorization for performance-critical vector kernels. Instead, these kernels use explicitly controlled vector code, including hand-written assembly, so the compiler emits only instructions supported by the target Spatz configuration. A future compiler flow could enable more automatic vectorization by exposing the supported Spatz instruction subset to the compiler or by otherwise constraining RVV code generation accordingly.

5.2 Topology Discovery and Core Identification

The runtime exposes topology-query functions that allow software to identify the position of the executing hart within the CachePool hierarchy. These functions abstract the compile-time system configuration and allow the same application code to run across configurations with different numbers of Groups, Tiles, scalar cores, and vector units.

The runtime provides identifiers and counts at several hierarchy levels, including the global core index, cluster-local core index, Tile index, Group index, and the corresponding number of resources. In addition, helper functions report the number and index of physical Spatz vector units, which differ from the hart count in the 2S1V configuration because two Snitch harts share one Spatz instance. The function `snrt_cluster_is_primary()` identifies the default owner of a shared Spatz pair and is used by software targeting dual-scalar Core Complexes.

Representative topology functions include

- `snrt_global_core_idx()` and `snrt_global_core_num()` for system-wide core identification;
- `snrt_cluster_core_idx()` and `snrt_cluster_core_num()` for logical-cluster-local identification;
- `snrt_cluster_tile_idx()` and `snrt_cluster_tile_num()` for Tile-level topology information;

- `snrt_cluster_group_idx()` and `snrt_cluster_group_num()` for Group-level topology information;
- `snrt_cluster_vpu_idx()` and `snrt_cluster_vpu_num()` for physical Spatz identification.

These helpers are used throughout the runtime and application kernels to derive work partitions, select local memory regions, construct barrier masks, and map software tasks onto the physical hierarchy.

5.3 Memory Allocation and Placement

Dynamic memory allocation in CachePool is performed in off-chip memory, while allocated data are transparently cached in the shared L1 according to the configured address-mapping and partitioning policies.

The runtime provides a lightweight DRAM allocator through `snrt_malloc()` and `snrt_free()`. The allocator uses a linked-list organization in DRAM and supports block coalescing on deallocation. Both allocation metadata and payloads are aligned to the 64 B cache-line size, and requested payload sizes are rounded up to a cache-line multiple.

The allocator is intentionally not thread-safe and is expected to be used during single-core initialization phases. A typical application therefore allocates and initializes shared data structures on one designated core before synchronizing the participating cores and entering the parallel computation phase.

Once allocated, data are accessed through the normal CachePool memory hierarchy. Their effective placement within the distributed L1 is determined by the programmable address-interleaving and cache-partitioning mechanisms described in the following subsections.

In addition to dynamically allocated data, each Snitch core uses a dedicated 1024 B local stack SRAM for low-latency stack accesses. When the local stack region is insufficient, excess stack storage is mapped into a core-specific region of the shared L1 address space, as described in Section 3.2.4.

5.3.1 L1 Address Scrambling

The runtime exposes software control over the L1 address-scrambling mechanism through `l1d_xbar_config()`. The argument specifies the crossbar interleaving offset in address bits and therefore controls the granularity at which consecutive memory regions are distributed across L1 destinations.

The runtime enforces a minimum granularity equal to the cache-line size so that all bytes belonging to the same cache line remain mapped to one cache controller. For the default 64 B cache line, an offset of 6 corresponds to cache-line-granular interleaving.

Changing the scrambling configuration allows software to trade locality against aggregate bandwidth. Streaming kernels can use finer-grained interleaving to distribute traffic across more banks and Tiles, while kernels with strong spatial reuse can use coarser mappings to retain more accesses within a local region.

5.3.2 Cache Partitioning and Address Classification

The runtime exposes the programmable private/shared L1 partition through `l1d_part()` and `l1d_part_folded()`. The partition size specifies the number of private banks per Tile, ranging from zero for a fully shared configuration to four for a fully private configuration.

The default `l1d_part()` interface selects Tile-private behavior for banks assigned to the private region. The extended `l1d_part_folded()` interface additionally allows software to select Group-private operation through the `L1D_FOLD_GROUP` mode. In this mode, the private partition

is shared among the Tiles of one Group rather than replicated independently per Tile, while the shared partition remains globally visible across the system.

Group folding is particularly useful for data that is reused by all Tiles within one Group. Instead of fetching one copy per Tile, the Group can share a single cached copy through the intra-Group network. The trade-off is that accesses to nominally private data may target a sibling Tile within the same Group and therefore incur an intra-Group access rather than a purely local Tile access.

The address boundary between the shared and private regions is configured through `l1d_addr()`. In the current runtime, addresses below the configured boundary are mapped to the globally shared region, while addresses at or above the boundary are mapped to the private region. Changing either the partition organization or the address boundary requires invalidating or flushing resident cache state before the new mapping becomes active.

5.3.3 Cache Maintenance

The runtime provides cache-maintenance functions for full-cache, shared-region, and private-region operations. Application code normally uses the cluster-wide variants, which coordinate all participating cores before issuing the maintenance request.

The functions `l1d_cluster_flush()`, `l1d_cluster_shared_flush()`, and `l1d_cluster_private_flush()` respectively flush the complete L1, only the globally shared banks, or the private banks associated with selected Tiles. The cluster-wide functions internally execute a memory fence and hardware barriers, perform the actual maintenance operation on one designated core, and synchronize again before returning.

Low-level single-core variants are also provided for runtime initialization and internal use. The function `l1d_wait()` polls the hardware status register until the selected maintenance operation has completed.

Cache reconfiguration operations that alter the partitioning mode or the private/shared address boundary implicitly require a cold transition. The runtime therefore flushes the cache before applying such changes so that valid data cannot remain associated with an obsolete address mapping.

5.4 Synchronization and Work Distribution

CachePool provides several complementary synchronization mechanisms ranging from local memory ordering to system-wide collective barriers. The runtime exposes memory fences, atomic operations, software barriers, and the hierarchical hardware barrier described in Section 3.10.2.

5.4.1 Memory Ordering and Fences

The function `snrt_fence()` drains both outstanding Snitch scalar-memory operations and outstanding Spatz memory requests before subsequent instructions are allowed to proceed. This operation is required before synchronization points when earlier scalar or vector stores must be globally visible.

The runtime additionally provides `snrt_fence_snitch()` and `snrt_fence_spatz()` when only the scalar or vector memory path needs to be drained. These more selective fences can reduce unnecessary waiting in kernels that use only one of the two memory-issuing engines.

5.4.2 Hardware and Software Barriers

The default `snrt_cluster_hw_barrier()` function invokes the full hardware barrier on barrier slot 0 and synchronizes all participating cores in the logical cluster. The runtime also retains a polling-based software barrier and a higher-level global barrier for cases where a hardware collective is not appropriate.

More selective synchronization is supported through partial and Group-level barrier APIs. The function `snrt_cluster_partial_barrier()` restricts the participating cores within each Tile, while `snrt_cluster_group_barrier()` additionally allows software to select participating Tiles and optionally terminate synchronization at the Group level. A barrier ID selects one of the independent hardware barrier slots, allowing multiple synchronization contexts to remain active concurrently.

The function `snrt_barrier_set_group_mask()` configures the Group participation mask associated with a barrier slot for synchronization rounds that reach the top-level barrier. All participants in a given round must use identical masks and barrier identifiers to preserve a consistent synchronization domain.

5.4.3 Atomic Operations

The runtime exposes RISC-V atomic operations for fine-grained synchronization through the scalar Snitch memory path. For example, `snrt_mutex_lock()` and `snrt_mutex_release()` implement a blocking mutex using `amoswap.w` operations with acquire and release semantics.

Atomics complement the collective hardware barrier by providing synchronization for shared data structures and irregular producer-consumer interactions where a full collective synchronization point would be unnecessarily expensive.

5.4.4 Work Distribution

Parallel kernels derive work assignments from the topology-query functions described in Section 5.2. The global or local core identifier is commonly used to divide loop iterations or data partitions across cores, while Tile and Group identifiers can be used when the mapping should follow the physical memory hierarchy.

This explicit topology-aware work distribution allows software to align computational ownership with cache placement and interconnect locality. For example, kernels can assign data partitions to cores within one Group when Group-private cache partitioning is enabled, or distribute off-chip traffic across Groups and HBM channels according to the memory-placement policy.

5.5 Spatz Control

In the 2S1V configuration, two Snitch harts share one Spatz vector coprocessor and software must therefore coordinate vector-unit ownership. The runtime exposes blocking and non-blocking ownership primitives that operate on the hardware sharing mechanism described in Section 3.2.

The function `spatz_lock_acquire()` repeatedly attempts to acquire ownership until the request is accepted, while `spatz_lock_release()` relinquishes ownership after the current hart has completed its vector work. The underlying hardware operation itself is non-blocking and returns one of three outcomes: failure due to contention, immediate success, or success with a pending ownership transition.

When the hardware reports a pending transition, the runtime returns without waiting for the previous owner's Spatz instructions to fully drain. The accelerator multiplexer prevents the new owner from issuing vector operations until the transition completes, so the first subsequent vector or floating-point instruction naturally stalls until ownership is safe.

For applications requiring finer control, `spatz_lock_try_acquire()` and `spatz_lock_try_release()` expose the non-blocking ownership attempts directly. On 1S1V configurations, these APIs reduce to harmless operations because each Snitch already has exclusive access to its Spatz instance.

The runtime also provides host-specific barrier helpers for dual-scalar configurations. The functions `snrt_cluster_host0_barrier()` and `snrt_cluster_host1_barrier()` synchronize

only the corresponding members of each shared-Spatz pair and are implemented using the programmable hardware-barrier infrastructure.

5.6 Boot, Execution, and Profiling Support

The runtime provides the low-level startup and control functionality required to bring all Snitch harts into a consistent execution state before entering the application. At reset, one boot hart performs platform and runtime initialization while the remaining harts wait until they are released into the common application entry point.

The runtime initializes the topology descriptors, interrupt infrastructure, allocator state, and CachePool-specific memory subsystem before parallel execution begins. The L1 cache is invalidated during startup so that applications enter `main()` with a clean cache state and do not need to perform explicit cache initialization.

Memory-mapped peripheral registers additionally provide software-visible execution status, kernel cycle bookkeeping, boot-entry control, and end-of-computation signaling. These mechanisms are primarily used by the runtime, test infrastructure, and performance-evaluation environment rather than by individual kernels.

Together, the runtime abstractions provide a lightweight software layer over the configurable CachePool hardware while still exposing topology and memory-placement information when applications require architecture-aware optimization.

6 GVSoC Simulation Framework

To complement RTL simulation, we developed a high-level CachePool model based on GVSoC. GVSoC is a C++-based full-platform simulation framework designed for fast and configurable simulation of RISC-V systems. The CachePool integration provides a cycle-approximate model of the manycore architecture and is intended to support rapid architectural exploration, software development, and end-to-end workload evaluation at system sizes that are expensive to simulate in RTL.

The CachePool GVSoC target models the hierarchical organization of the architecture, including Core Complexes, Tiles, Groups, the distributed shared L1 data cache, the fine-grained L1 access network, and the cache-refill path. The model preserves the software-visible interfaces of the RTL system and executes the same CachePool ELF binaries without recompilation. This allows kernels and runtime software to move directly between RTL simulation and GVSoC, while providing a substantially faster platform for functional validation and performance exploration.

A major advantage of the GVSoC model is its configurability. System dimensions such as the number of Groups, Tiles per Group, Core Complexes per Tile, scalar cores per Core Complex, L1 cache banks, and memory-system parameters can be modified without rebuilding the application software. The simulator can therefore be used to explore architectural scaling and workload mapping before committing to expensive RTL implementation and physical-design iterations.

6.1 CachePool System Model

The GVSoC model follows the same hierarchical decomposition as the RTL implementation. A system contains a two-dimensional mesh of Groups, each Group contains multiple Tiles, and each Tile contains several Core Complexes together with a physically distributed slice of the shared L1 cache.

Each Core Complex contains one Spatz vector unit and one or two Snitch scalar cores depending on the selected configuration. The model also represents the distributed L1 cache controllers, local Tile-level crossbars, inter-Tile and inter-Group communication, and the dedicated cache-refill network.

Timing is modeled at cycle-level granularity using component-specific pipeline delays, resource contention, and memory-response latency. The goal is not to reproduce every RTL pipeline signal, but to preserve the dominant architectural timing effects required for system-level performance exploration.

TODO: Add detailed component diagram and describe the mapping between GVSoC components and the

6.2 Configurable Architecture Exploration

Unlike the RTL flow, where major topology parameters are fixed during elaboration, the GVSoC target exposes the system topology through run-time configuration. The number of Groups in each mesh dimension, the number of Tiles per Group, the number of Core Complexes per Tile, the number of scalar cores sharing each Spatz, the number of L1 banks, and the memory latency can therefore be changed without recompiling the application binary.

This enables rapid exploration of scaling trends across different CachePool organizations. For example, the same kernel binary can be evaluated on configurations ranging from a single Group to larger two-dimensional Group meshes, provided that the workload itself supports the selected number of participating cores.

TODO: Add a compact table of the principal GVSoC configuration parameters and their correspondence

6.3 Memory-System Modeling

The CachePool GVSoC model supports multiple levels of memory-system fidelity. The default configuration uses a fixed-latency backing memory behind the cache-refill network, providing fast

simulation for architectural and software exploration. The memory latency is configurable and can be calibrated against RTL measurements for workloads whose performance is dominated by L1 behavior.

For more detailed off-chip memory studies, GVSoC can also be coupled with DRAMSys. In this configuration, individual memory channels are modeled using separate DRAM instances and accesses are interleaved across the channels according to the configured address mapping. This mode captures timing effects that are absent from the fixed-latency model, at the cost of substantially longer simulation time.

TODO: Describe the final DRAMSys configuration, channel mapping, and supported HBM/DRAM orga

6.4 Software Compatibility and End-to-End Simulation

The GVSoC model preserves the software-visible address map and execution interfaces of CachePool. Consequently, the same ELF binaries generated for RTL simulation can be executed directly in GVSoC without recompilation.

This compatibility allows the complete CachePool software stack, including boot code, runtime support, synchronization primitives, cache configuration, Spatz control, and application kernels, to be exercised in the high-level model. GVSoC therefore serves not only as a performance model, but also as an end-to-end software-development and validation platform.

The simulator additionally exposes architectural statistics and tracing facilities that can be used to analyze cache activity, vector instruction issue, synchronization behavior, and memory-system traffic.

TODO: Add the final set of statistics and traces used in the evaluation flow.

6.5 Model Calibration and RTL Validation

The GVSoC model is calibrated against the RTL implementation using identical application binaries and matching architectural configurations. Cycle counts measured over application work regions are compared rather than complete simulation runtimes, because the two simulation environments use different mechanisms for loading program and data images.

Initial validation indicates that workloads whose working sets remain primarily within the L1 cache can be reproduced with relatively small timing error. Larger deviations are observed for strongly refill-bandwidth-bound kernels, where simplified outstanding-miss and backing-memory models have a greater influence on performance.

The model is therefore primarily intended for fast architectural exploration and relative design-space comparisons, while RTL simulation remains the reference for cycle-accurate evaluation.

TODO: Insert the final RTL–GVSoC validation results and error statistics.

6.6 Current Limitations

The GVSoC model intentionally abstracts several RTL implementation details in order to maintain high simulation speed. In particular, the accuracy of refill-bound workloads depends on the selected cache-miss and backing-memory model, and the fixed-latency memory configuration does not reproduce channel-level DRAM timing effects.

Some architectural mechanisms are also modeled at a higher level than their RTL counterparts. These approximations are acceptable for early architectural exploration but must be considered when interpreting absolute cycle counts.

TODO: Partner: document the remaining differences between the current GVSoC model and RTL, inclu

7 Kernel Characterization

This section describes the software implementation of key micro-kernels used in the ManyRVData dataplane. At the current stage, the algorithm is decomposed into multiple standalone micro-kernels to simplify implementation and enable cross-comparison. In the second half of the project, these kernels will be chained and evaluated to represent the full control algorithm.

The main micro-kernels are:

- **Dot Product (DOTP)**: Computes the dot product between two vectors, producing a scalar result.
- **Matrix-Vector Multiplication (MVM)**: Computes the product of a matrix and a vector, producing a vector result.
- **Matrix Multiplication (MatMul)**: Computes the product of two matrices, producing a matrix result.
- **Shannon–Hartley Channel Capacity Formula (Shannon)**: Evaluates $\vec{y} = \log(1 + \vec{x})$, producing a vector result.
- **Vector Sorting**: Sorting an input vector from largest to smallest.

7.1 DOTP

This kernel is implemented using vector instructions and distributed across vector cores according to their core IDs. Two input vectors are loaded into the vector core and processed using vector-vector multiply-accumulate (`vmacc`) operations. The partial results are first reduced within each vector core, followed by a final scalar reduction across cores.

7.2 MVM

This kernel is implemented using vectorized column-wise scalar–vector multiplications. Each column of the input matrix is loaded via vector load instructions and multiplied by the corresponding scalar element from the input vector, with results accumulated across the column. Each core contributes a portion of the final output vector.

7.3 MatMul

This kernel is based on the Gustavson algorithm, which streams scalar elements from matrix A and performs multiply-accumulate operations with columns of matrix B. The approach overlaps vector load latencies with computation, enabling sustained throughput, and distributes workloads across cores with minimal interference.

In practice, each row of matrix A is streamed element by element, and the corresponding scalar values are broadcast across the vector cores. Each scalar multiplies an entire column of matrix B, which is held in vector registers, followed by an accumulation into the partial results of matrix C. This streaming style reduces memory traffic by reusing data from matrix B across multiple iterations, while keeping the vector units busy.

The Gustavson algorithm is particularly attractive for the CachePool architecture: It exposes a high degree of parallelism across both rows of matrix A and columns of matrix B, maps naturally to vector–scalar and vector–vector multiply-accumulate instructions, and avoids inter-core synchronization during the main loop. The resulting workload distribution ensures balanced utilization of compute and memory resources, making this kernel compute-bound rather than memory-bound at large problem sizes.

7.4 Shannon

The Shannon kernel is motivated by the Shannon–Hartley channel capacity formula, which characterizes the maximum achievable data rate of a communication channel as

$$C = B \cdot \log_2 \left(1 + \frac{S}{N} \right),$$

where B is the channel bandwidth, S is the signal power, and N is the noise power. The logarithmic term $\log(1 + S/N)$ appears frequently in physical-layer and baseband algorithms, making it an important target for acceleration.

Within the control algorithm, this micro-kernel is applied to compute the subcarrier gain for each user. Here, the bandwidth factor can be omitted, the signal-to-noise ratio is represented as a vector input, and the logarithm is evaluated in the natural base. The kernel therefore simplifies to

$$\vec{G} = \log(1 + \vec{S}),$$

where $\vec{G}$ denotes the per-subcarrier gains and $\vec{S}$ is the vector of signal-to-noise ratios produced by a previous stage.

A practical implementation will approximate the natural logarithm using polynomial expansions, such as a Taylor series or minimax approximation, to strike a balance between accuracy and computational efficiency. These approximations can be mapped efficiently to vector instructions, enabling parallel evaluation across all input elements.

7.5 Vector Sorting

The vector sorting kernel targets a straightforward ordering operation, producing a vector rearranged from maximum to minimum. Unlike indirect sorting, which maintains a separate index vector, this kernel focuses on directly reordering the data elements.

A key challenge arises from the lack of dedicated sorting instructions in the RVV 1.0 ISA. Consequently, sorting must be constructed from existing primitives such as masking, gathering, and indexed memory operations. On the Spatz core, masking and gather functionality are still under development, and scatter operations are not yet available. As a potential workaround, indexed load and store instructions could be used to emulate scatter–gather behavior.

Alternatively, the scalar Snitch core could be employed to execute the sorting algorithm, trading off vector parallelism for reduced implementation complexity.

The detailed design and evaluation of both the vector sorting and Shannon kernels will be carried out in the next phase of the project.

8 RLC/MAC Dataplane Implementation

This section presents the software implementation of the RLC kernels used in the *ManyRVData* dataplane. We focus on downlink AM-mode processing at gNodeB side under high-load traffic models, aligned with the Layer-2 DP scenarios and timing target provided by the partner specifications. In our DP setting, RLC entities operate under tight TTI constraints (typ. 0.5 ms), must sustain multiple million packets per second throughput, and manipulate irregular data structures (linked lists), yielding sparse and non-contiguous memory accesses. These properties favor a cache-based, hardware-managed memory hierarchy rather than software-managed DMA and scratchpad memory. *ManyRVData* therefore targets a shared-cache cluster. By default, L1 cache banks are shared across all cores, with an optional per-tile private L1 partition for stronger locality or isolation.

8.1 Workload model and constraints

We adopt three traffic scenarios for evaluation (single-user peak, multi-user peak, multi-user average). They determine per-slot packet counts per RLC entity, and consequently the concurrency and memory pressure seen by the kernels. The partner document defines packet sizes, per-second slot counts, and closed-form relationships between throughput and packet rate, which we reuse in our test harness.

Table 7: RLC evaluation scenarios (DL/UL). Values shown are from the partner’s L2DP description and serve as our default harness configuration.

Scenario	Users	DL Gbps	DL pkt B	DL pps	DL slots/s	DL pkts/slot	UL Gbps	UL pkts/slot
Single-user peak	1	20	1350	1,851,851	1600	1157	1	1953
Multi-user peak	48	50	800	7,812,500	1600	4882	8	15625
Multi-user average	4800	20	800	3,125,000	1600	1963	4	7813

Notes. Per-user rates derive from the total divided by the number of users; slot counts per second are fixed by numerology; TB sizes for assembly come from the control/scheduler stage and are provided to the RLC assembly task at runtime.

Protocol and configuration. All entities run in Acknowledged Mode (AM) with 18-bit sequence numbers (SN); `pollPDU=32` and `pollByte=25,000`. In our testcases, gNB receives only *positive* ACK STATUS PDUs; retransmission and timers are disabled.

8.2 Software Architecture and Implementation

8.2.1 Workflow of the RLC Kernel

The RLC kernel can be abstracted as a producer–consumer program: producers generate and enqueue work items; the consumer processes them and maintains protocol state. The end-to-end workflow and how the two sides interact are shown in Fig. 12.

Producers repeatedly pull the next PDCP SDU descriptor from a shared cursor (under a small lock), request a `Node` from the fixed-size pool (`mm_alloc()` under allocator lock), fill its metadata (`data`, `tgt`, `len`), and append it to the tail of the shared *to-send* list (`list_push_back()` under list lock). If allocation fails or no new SDU is available, the producer enters the *Wait* state (short backoff) and retries.

A single *consumer* thread loops on the *to-send* list: it removes the head node (`list_pop_front()` under lock); if the list is empty, it briefly *Waits*. Otherwise, it performs the data move (vectorized memcpy standing in for package assembly) from source memory address to the target address, then updates RLC status: increments `pduWithoutPoll` and `byteWithoutPoll`, advances the

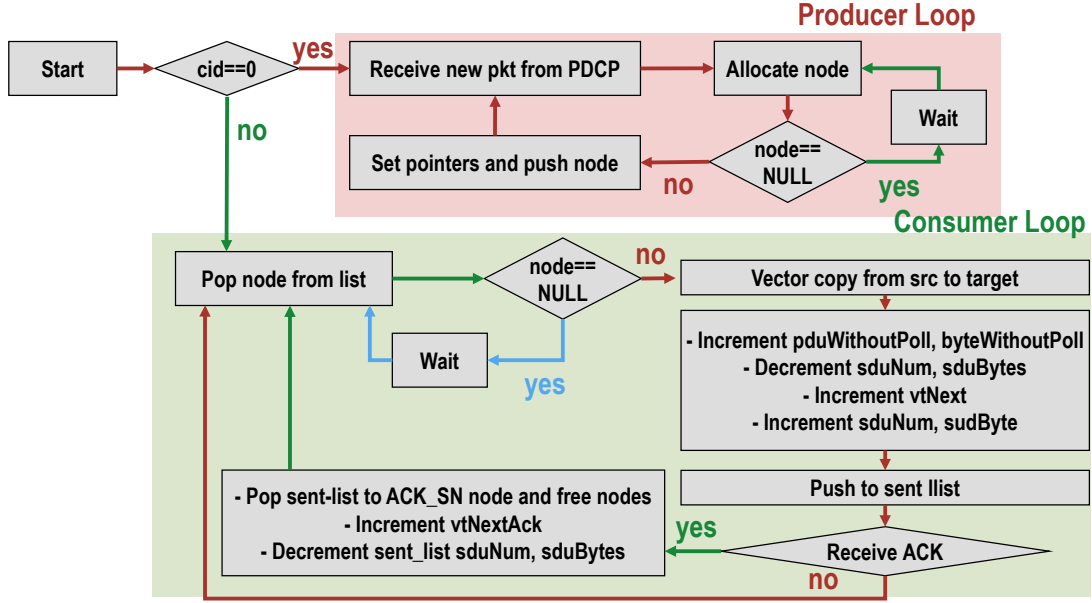


Figure 12: High-level producer-consumer architecture for RLC kernel

next sequence number `vtNext`, and accounts the package leaving/entering queues by decrementing `sduNum/sduBytes` on the *to-send* list and incrementing the same counters on the *sent* list. The processed node is appended to the *sent* list under lock. Upon reception of a STATUS with `ACK_SN`, the consumer pops acknowledged nodes from the head of the *sent* list up to `ACK_SN`, advances `vtNextAck`, decrements the *sent*-list counters accordingly, and returns each acknowledged node to the pool via `mm_free()`.

All shared structures (PDCP cursor, allocator, both linked lists) are protected by compact spinlocks to ensure atomic push/pop and allocation; a one-time cluster barrier precedes the workflow so producers and consumer start from a consistent state. Back-pressure naturally arises at two points shown as *Wait*: empty queues (consumer idle) and pool exhaustion (producers back off). The loop continues until producers exhaust PDCP SDUs and the consumer drains and acknowledges all in-flight nodes.

8.2.2 Producer-Consumer Roles

Producers. Producer threads simulate packet ingress (e.g., PDCP SDUs) and enqueue work items into a shared *to-send* queue implemented as a doubly linked list. Each producer repeatedly: (i) obtains the next PDCP packet descriptor from a shared cursor, (ii) allocates a `Node` from a fixed-size memory pool, (iii) fills its metadata (payload pointer, size), and (iv) appends it to the tail of the *to-send* list under a lock. This models concurrent ingress sources feeding the RLC layer.

Consumer. A single consumer thread represents the RLC transmitter. It repeatedly: (i) pops a `Node` from the head of the *to-send* list under a lock, (ii) performs data movement (copy/assemble) to the target buffer (standing in for AMD PDU assembly), (iii) updates RLC state (e.g., sequence counters; bytes/PDUs without poll), and (iv) pushes the `Node` into a *sent* list (tracking in-flight PDUs) under a lock. The consumer also simulates ACK processing: once an in-flight threshold is reached, it dequeues a batch from the *sent* list (front), advances the transmit window, and returns those `Nodes` to the memory pool.

8.2.3 Concurrency and Synchronization

Spinlocks. Shared resources (*to-send* list, *sent* list, memory pool, and the PDCP package cursor) are protected by simple spinlocks implemented with atomic test-and-set. There is also an optional backoff delay added to reduce contention. A hardware barrier synchronizes cores once after initialization to ensure a consistent start.

8.2.4 Data Structures and Memory Management

Double linked lists. Work queues are linked lists (head/tail, counters) of nodes. `list_push_back()` appends in $O(1)$ time (updates tail), `list_pop_front()` removes in $O(1)$ time (updates head).

Node. Each `Node` holds `prev/next` pointers, a pointer to the source payload, a pointer to the destination buffer, and the payload length.

Fixed-size pool allocator. A global memory context manages a statically allocated, cacheline-aligned buffer partitioned into fixed-size *pages* (one page per `Node`). Allocation uses a lock-protected bump pointer; when the pool is exhausted, freed pages are recycled from a lock-protected free-list. This yields deterministic, fragmentation-free allocation without a general-purpose heap.

8.2.5 Task Scheduling and Thread Management

For now, we use a static, bare-metal mapping: each core executes one function (producer or consumer). Coordination is via shared queues and spinlocks. Back-pressure arises naturally from the finite memory pool and queue occupancy.

9 Experimental Methodology

TODO: Consolidate all assumptions needed to interpret the results: evaluated system configurations, RTL/s

9.1 Evaluated System Configurations

TODO: Add configuration table.

9.2 Simulation and Memory-System Setup

TODO: Add RTL simulator and HBM/DRAM model details.

9.3 Benchmarks and Metrics

TODO: Define workload sizes and reported metrics.

10 Evaluation

This section evaluates the performance of the CachePool single-tile cluster through cycle-accurate simulation using QuestaSim, with *DRAMSys* serving as the main memory model. We investigate the impact of different cacheline widths (128 b, 256 b, and 512 b) under both hot- and cold-cache conditions. The simulations are carried out on the configurations with FPU.

10.1 Vector Kernel Benchmarking

To better illustrate the performance characteristics of the design, hardware utilization is analyzed through both cycle-level measurements and the roofline model.

Figure 14 reports the execution cycles of representative vector kernels. Solid bars correspond to hot-cache runs, while the shaded extensions capture the additional cycles incurred under cold-cache conditions. In general, longer cachelines improve performance, particularly in the cold-cache case.

For the most memory-bound kernel (dot product), the 512 b configuration achieves over 20% FPU utilization even under cold-cache conditions, highlighting the cluster’s ability to tolerate memory latency and exploit available off-chip bandwidth. As data reuse increases, cache misses have a smaller performance impact. For compute-bound kernels, such as matrix multiplication, the cluster reaches over 90% FPU utilization in both hot- and cold-cache scenarios, demonstrating that computation resources can be fully exploited when memory bandwidth is not the limiting factor.

In addition to the execution cycles and utilization data shown in Figure 14, Figure 13 presents roofline plots that relate achieved performance to the theoretical compute and memory bandwidth limits. A consistent trend can be observed: in hot-cache scenarios, the cluster performance approaches the roof, indicating that there are no significant bottlenecks within the core microarchitecture or the core-to-L1 interconnect for typical vector kernels. In cold-cache runs, performance progressively converges toward the hot-cache results as the arithmetic intensity increases. Moreover, the performance gap between hot and cold caches narrows with larger cacheline widths, owing to the prefetching benefits of longer cachelines that improve hit rate and bandwidth utilization.

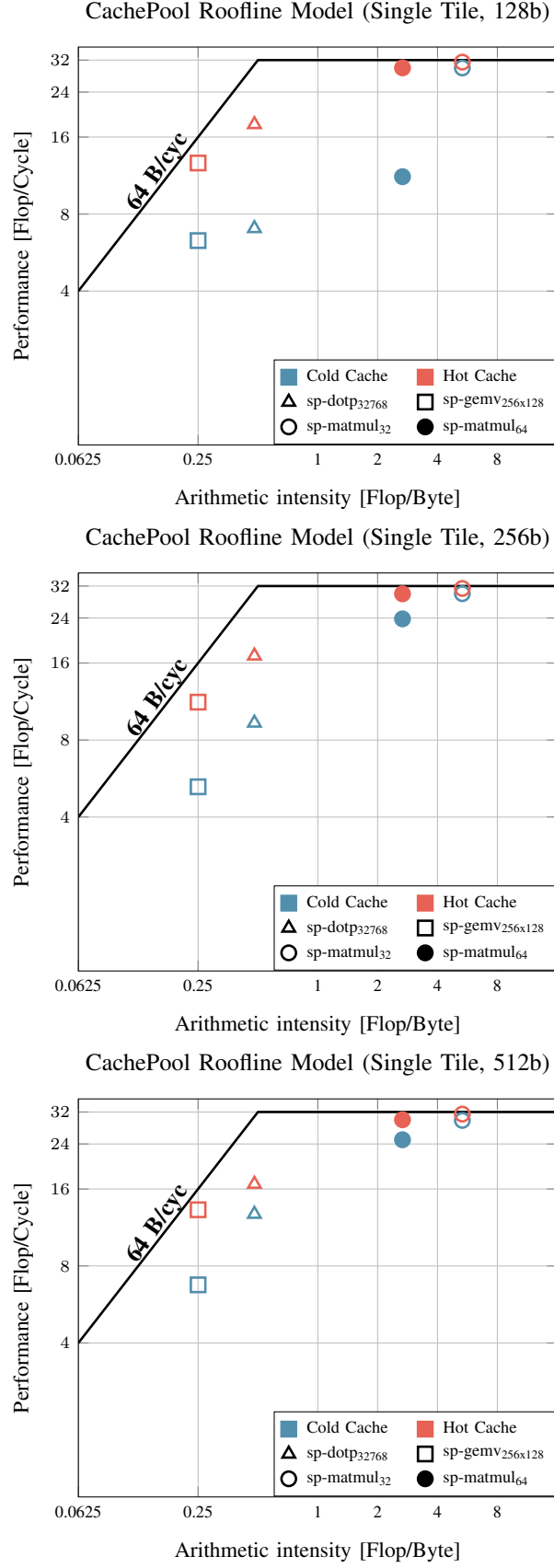


Figure 13: Roofline plots of the single-tile CachePool cluster with 128-, 256-, and 512-bit cachelines. Each plot shows the achieved performance of representative vector kernels under hot- and cold-cache conditions relative to the theoretical compute and bandwidth roofs.

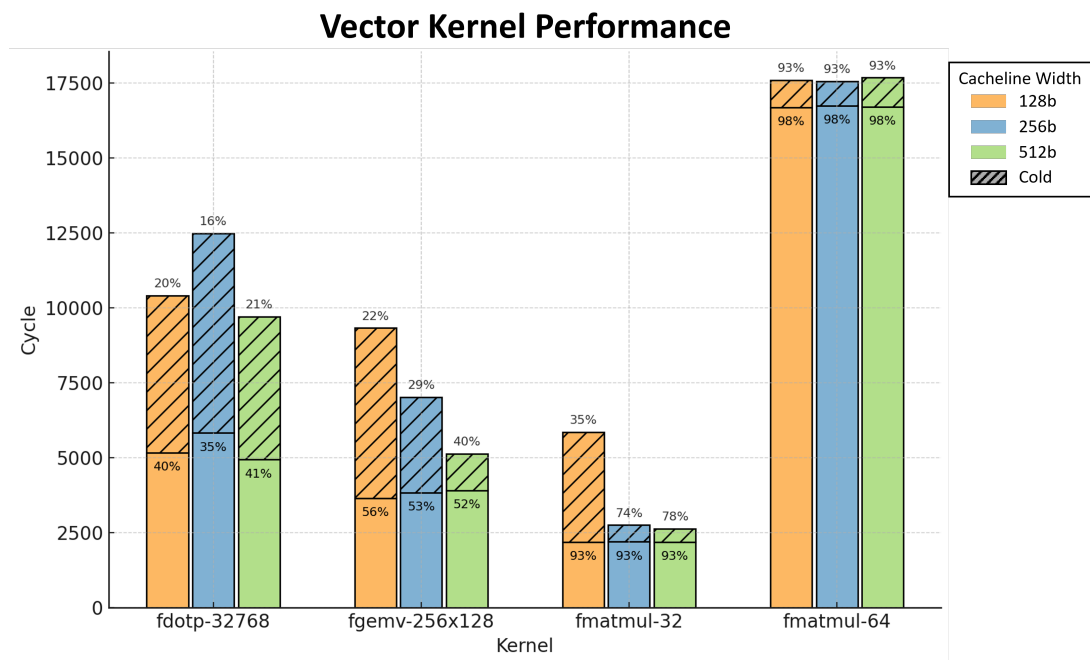


Figure 14: Vector kernel benchmark results of the CachePool cluster with different cacheline widths. Solid bars represent hot-cache execution, shaded extensions show additional cycles for cold-cache execution.

10.2 RLC Kernel Benchmarking

The performance results presented in Table 8 and Table 9 offer an initial view of the RLC data processing kernel performance on the current single tile implementation and how cacheline width impacts them. In this evaluation, we assume the clock frequency is 1 GHz.

Table 8: Data Movement Kernel for One 1350-Byte RLC Package

Cacheline Width	Cycle/Pkg	Bandwidth (Gbps)
128-bit	330	30.48
256-bit	259	38.84
512-bit	243	41.39

In Table 8, which evaluates the Data Movement Kernel responsible for loading, assembling, and storing a single 1350-byte RLC packet using vector instructions, wider cachelines clearly lead to better memory bandwidth utilization. The 512-bit cacheline configuration achieves the highest throughput (41.39 Gbps), indicating that vector memory operations benefit significantly from longer cachelines due to longer bursts and data prefetching.

Table 9: Full RLC Data Management Kernel

Cacheline Width	Cycle/Pkg	BW (Gbps)	Avg L1\$ Miss Rate	Avg Miss Stall Cycle
128-bit	1552	6.48	59.8%	132 667
256-bit	620	16.22	44.4%	2 448
512-bit	1051	9.57	43.9%	47 848

However, this trend does not hold in Table 9, which measures the Full RLC Data Management Kernel, a more comprehensive workload including scalar control logic, linked list traversal, and producer-consumer synchronization. The results reveal that the 256-bit cacheline configuration offers the best performance, achieving a throughput of 16.22 Gbps with a cycle-per-package count of only 620. Interestingly, while both the 256-bit and 512-bit configurations show similar average L1 miss rates (44.4% vs. 43.9%), the 512-bit configuration suffers a dramatic increase in average miss stall cycles per cache bank. This stark contrast highlights the hidden cost of wider cachelines.

A deeper analysis suggests that the 512-bit configuration, due to its reduced number of cachelines under fixed total cache capacity, experiences more conflict misses and evictions. These evictions introduce higher memory channel pressure, blocking subsequent vector loads and degrading overall performance. In contrast, the 256-bit configuration strikes a balance between spatial locality and miss penalty, avoiding excessive evictions while maintaining high bandwidth utilization.

To mitigate such issues in future iterations, we plan to add hardware support for non-cacheable memory access, specifically targeting stream-like RLC packet data. This approach aims to prevent cache pollution by stream data and alleviate contention-induced stalls. Overall, our findings emphasize that wider cachelines do not necessarily translate to higher performance and must be carefully tuned based on workload characteristics.

It is important to emphasize that these results represent a preliminary evaluation. Both the hardware and software stacks are still evolving. While these measurements serve as useful early indicators, further profiling is required to fully understand the bottlenecks behind each configuration. We plan to analyze pipeline stalls, cache behavior, and load/store throughput to guide architectural refinements.

Based on the current observations, we estimate that a 2-tile configuration should be capable of handling the single-user peak rate test case. For the more demanding multi-user average scenario, 4 to 6 tiles are likely sufficient. As of now, only the single-user test case has been

implemented and evaluated. Future work will include the implementation and performance analysis of both the multi-user average and peak workloads to validate system scalability and efficiency.

11 Discussion

TODO: Synthesize the architectural findings across experiments: globally shared versus private clustering, a

12 Future Work

TODO: The text below is inherited from the first-year report and must be revised against the current imple

Building on the progress made in the first year, we outline the following technical objectives and development tasks for the second year of the ManyRVData project.

12.1 Data Cache Optimization

- **Enhance Area Efficiency:** Optimize the internal architecture of the cache controller and the dimensions of data SRAM macros to improve the area efficiency. Improvements will focus on data RAM utilization and buffer sizing efficiency.
- **Reduce Load-to-Use Latency:** Refine the cache coalescer logic and minimize interconnect latency to reduce memory access latency, especially under contention.
- **Uncached Access Support:** Enable uncached request support to allow better handling of streaming data and reduce interference with frequently reused cache lines.
- **Prefetching Scheme Exploration:** Extend beyond the current next-3-line prefetching (enabled by a 512-bit cacheline design) to investigate more adaptive or pattern-based prefetching strategies.

12.2 Data Cache Partition Exploration

Design and implement configurable tile-private L1 cache partitioning to support high-bandwidth, low-latency local access in multi-tile systems. We plan to explore the following two designs, evaluate their performance and PPA impact:

- **Bank-level Partitioning:** A scheme where the whole cache bank can be configured as a private partition. This scheme is simpler to implement, but has a coarser partition granularity, and the bandwidth for both the shared and private partitions will be reduced.
- **Set-index-level Partitioning:** A finer-grain approach that enables higher concurrency by partitioning the cache based on the set index. This scheme is more complex, but has a finer partition granularity and will not reduce the cache access bandwidth for all the partitions.

Software runtime support will be explored to help manage the partitions, especially the cache coherence between the private partitions.

12.3 Processing Element Microarchitecture Optimization

Further relax RAW hazard checks between the scalar and vector cores within each PE to enhance concurrent scalar-vector execution. This aims to improve dual-issue utilization and is particularly relevant for workloads with frequent scalar-vector interleaving.

12.4 Multi-Tile Scaling and Interconnect Exploration

Scale the architecture to support 4–8 tiles to accommodate multi-user, high-throughput workloads. Investigate and compare different inter-tile interconnects (e.g., hierarchical crossbar, mesh-based network-on-chip) in terms of performance, latency, PPA, and software programmability.

12.5 Advanced Backend Implementation and PPA Evaluation

Shift the backend implementation from the current 12nm technology node to a more advanced node. Do the complete back-end flow, including logic synthesis, floorplanning, placement, and routing, have a detailed PPA analysis and design iteration.

12.6 Software Kernel Finalization and Evaluation

- **RLC Data Management Kernel:** Extend current implementation to support multiple users and support the multi-tile design, as well as the runtime support for the configurable cache partition schemes.
- **Control Algorithm Kernels:** Implement and benchmark additional kernels with representative control logic patterns, including Shannon-Hartley Channel Capacity Calculation and Vector Sorting. These kernels will validate the efficiency of mixed scalar-vector execution under real-time DP constraints.

12.7 Exploration of Private L1 Cache

A parallel effort has been initiated to investigate the feasibility and performance impact of introducing a small private L1 Write-Through (WT)-cache for each core, hierarchically connected to the existing shared L2 cache subsystem. In this configuration, only lightweight coherence mechanisms are required, mainly invalidations triggered when an entry is updated in the L2 cache, thereby reducing protocol complexity compared to coherent designs for write-back L1 caches.

Building upon this concept, a lazy write-through policy is being explored. Unlike a conventional write-through cache that propagates every store immediately to the next level, the lazy write-through scheme temporarily retains dirty data in the L1 and propagates updates to the L2 cache either periodically or upon specific requests. This approach aims to balance the simplicity of write-through with the bandwidth efficiency of write-back, potentially reducing redundant writes and improving overall performance.

The effectiveness of this design will be evaluated against the current cluster architecture to identify the most efficient memory hierarchy for future system

13 Conclusion

TODO: Write after the architecture and evaluation sections are finalized. Summarize the final ManyRVDat

References

- [1] Samuel Riedel et al. “MemPool: A Scalable Manycore Architecture with a Low-Latency Shared L1 Memory”. In: *IEEE Transactions on Computers* 72.12 (2023), pp. 3561–3575. DOI: 10.1109/TC.2023.3307796. URL: <https://arxiv.org/abs/2303.17742>.
- [2] Yichao Zhang et al. “TeraPool: A Physical Design Aware, 1024 RISC-V Cores Shared-L1-Memory Scaled-up Cluster Design with High Bandwidth Main Memory Link”. In: *IEEE Transactions on Computers* (2025), pp. 1–14. DOI: 10.1109/TC.2025.3603692.
- [3] Ampere Computing. *AmpereOne Family Product Brief*. May 2023. URL: <https://amperecomputing.com/briefs/ampereone-family-product-brief> (visited on 09/15/2025).
- [4] AMD EPYC[®] 9004 Series Processors: Architecture Overview. Tech. rep. Advanced Micro Devices, Inc. (AMD), 2024. URL: <https://www.amd.com/system/files/documents/epyc-9004-series-processors-architecture-overview.pdf> (visited on 09/15/2025).
- [5] Intel Corporation. *The Next-Generation Efficient-core Xeon Processor (Sierra Forest)*. Hot Chips 2023 presentation. 2023. URL: <https://www.intel.com/content/www/us/en/events/hot-chips-2023/briefings.html?presentation=787431> (visited on 09/15/2025).
- [6] NVIDIA. *NVIDIA Grace Hopper Superchip Architecture In-Depth*. Nov. 2022. URL: <https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/> (visited on 09/15/2025).
- [7] Daniel Sorin, Mark Hill, and David Wood. *A primer on memory consistency and cache coherence*. Morgan & Claypool Publishers, 2011.
- [8] Milo MK Martin, Mark D Hill, and David A Wood. “Token coherence: Decoupling performance and correctness”. In: vol. 31. 2. ACM New York, NY, USA, 2003, pp. 182–193.
- [9] Xiangyao Yu and Srinivas Devadas. “Tardis: Time traveling coherence algorithm for distributed shared memory”. In: *2015 International Conference on Parallel Architecture and Compilation (PACT)*. IEEE. 2015, pp. 227–240.
- [10] Xiaowei Ren et al. “Hmg: Extending cache coherence protocols across modern hierarchical multi-gpu systems”. In: *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE. 2020, pp. 582–595.
- [11] Johnathan Alsop, Matthew Sinclair, and Sarita Adve. “Spandex: A flexible interface for efficient heterogeneous coherence”. In: *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE. 2018, pp. 261–274.
- [12] Xiangyao Yu et al. “Tardis 2.0: Optimized time traveling coherence for relaxed consistency models”. In: (2016), pp. 261–274.
- [13] Arm Ltd. *Memory attributes*. Available at: <https://developer.arm.com/documentation/100941/0101/Memory-attributes>. 2023. URL: <https://developer.arm.com/documentation/100941/0101/Memory-attributes>.
- [14] Arm Ltd. *AMBA AXI and ACE Protocol Specification*. AXI Coherency Extensions (ACE). 2013. URL: <https://developer.arm.com/documentation/ih10022/latest>.
- [15] Arm Ltd. *CoreLink CMN-600 Coherent Mesh Network - Technical Reference Manual*. Document Number: 100614. 2020. URL: <https://developer.arm.com/documentation/100180/latest/>.
- [16] HSA Foundation. *HSA System Architecture Specification v1.2*. Available at: <https://hsafoundation.com/wp-content/uploads/2021/02/HSA-SysArch-1.2.pdf>. 2020. URL: <https://hsafoundation.com/wp-content/uploads/2021/02/HSA-SysArch-1.2.pdf>.

- [17] Marc S Orr et al. “Synchronization using remote-scope promotion”. In: *ACM SIGARCH Computer Architecture News* 43.1 (2015), pp. 73–86.
- [18] Compute Express Link Consortium. *What is CXL?* <https://www.computeexpresslink.org/about-cxl>. Available at: <https://www.computeexpresslink.org/about-cxl>. 2023.
- [19] Rambus Inc. *Compute Express Link (CXL): All you need to know*. <https://www.rambus.com/blogs/compute-express-link/>. Available at: <https://www.rambus.com/blogs/compute-express-link/>. 2024.
- [20] Arm Ltd. *Accelerating Cloud Workloads with CCIX*. https://www.arm.com/-/media/global/events/techcon/2017/pdf/303_armtechcon2017_acceleratingcloudworkloads.pdf. Presented at Arm TechCon 2017. 2017.
- [21] Cadence Design Systems. *CCIX Verification Solution from Cadence*. https://community.cadence.com/cadence_blogs_8/b/ip/posts/ccix-verification-solution. Available at: https://community.cadence.com/cadence_blogs_8/b/ip/posts/ccix-verification-solution. 2019.
- [22] Rebecca Lewington. *An AI Chip With Unprecedented Performance To Do the Unimaginable*. Aug. 2021. URL: <https://www.cerebras.ai/blog/an-ai-chip-with-unprecedented-performance-to-do-the-unimaginable> (visited on 09/15/2025).
- [23] Yichao Zhang et al. “TeraPool-SDR: An 1.89 TOPS 1024 RV-Cores 4 MiB Shared-L1 Cluster for Next-Generation Open-Source Software-Defined Radios”. In: *Proceedings of GLSVLSI 2024*. 2024. DOI: 10.1145/3649476.3658735. URL: <https://arxiv.org/abs/2405.04988>.
- [24] Intel Labs. *The SCC Platform Overview*. Tech. rep. Intel Corporation, May 2010. URL: <https://www.intel.com/content/dam/www/public/us/en/documents/technology-briefs/intel-labs-single-chip-platform-overview-paper.pdf> (visited on 09/15/2025).
- [25] Kalray. *COOLIDGE™ MPPA® DPU Whitepaper*. Tech. rep. Kalray, 2022. URL: https://www.kalrayinc.com/wp-content/uploads/2023/10/WP_Kalray_MPPA_DPU_Coolidge_june2022.pdf (visited on 09/15/2025).
- [26] Xiaowei Ren et al. “HMG: Extending Cache Coherence Protocols Across Modern Hierarchical Multi-GPU Systems”. In: *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 2020, pp. 582–595. DOI: 10.1109/HPCA47549.2020.00056. URL: <https://ericrxw.github.io/xiaoweiren/docs/HPCA2020/HMG-HPCA2020.pdf>.
- [27] NVIDIA. *NVIDIA H100 Tensor Core GPU Architecture*. en-us. URL: <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>. URL: <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c> (visited on 09/30/2024).
- [28] Samuel Riedel et al. “MemPool: A Scalable Manycore Architecture with a Low-Latency Shared L1 Memory”. In: *IEEE Transactions on Computers* 72.12 (2023), pp. 3561–3575. DOI: 10.1109/TC.2023.3307796. URL: <https://arxiv.org/abs/2303.17742>.
- [29] Yichao Zhang et al. “TeraPool: A Physical Design Aware, 1024 RISC-V Cores Shared-L1-Memory Scaled-up Cluster Design with High Bandwidth Main Memory Link”. In: *IEEE Transactions on Computers* (2025), pp. 1–14. DOI: 10.1109/TC.2025.3603692.
- [30] Taehyun Kim et al. “Scalable Bandwidth Shaping Scheme via Adaptively Managed Parallel Heaps in Manycore-Based Network Processors”. In: *ACM Trans. Des. Autom. Electron. Syst.* 22.4 (2017). ISSN: 1084-4309. DOI: 10.1145/3065926. URL: <https://doi.org/10.1145/3065926>.

- [31] Yang Hu and Tao Li. “Enabling Efficient Network Service Function Chain Deployment on Heterogeneous Server Platform”. In: *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 2018, pp. 27–39. DOI: 10.1109/HPCA.2018.00013.
- [32] Bowen Wang et al. *A Dynamic Allocation Scheme for Adaptive Shared-Memory Mapping on Kilo-core RV Clusters for Attention-Based Model Deployment*. 2025. arXiv: 2508.01180 [cs.AR]. URL: <https://arxiv.org/abs/2508.01180>.
- [33] Scott Davidson et al. “The Celerity Open-Source 511-Core RISC-V Tiered Accelerator Fabric: Fast Architectures and Design Methodologies for Fast Chips”. In: *IEEE Micro* 38.2 (2018), pp. 30–41. DOI: 10.1109/MM.2018.022071133.
- [34] Dai Cheol Jung et al. “Scalable, Programmable and Dense: The HammerBlade Open-Source RISC-V Manycore”. In: *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024, pp. 770–784. DOI: 10.1109/ISCA59077.2024.00061.
- [35] Dai Cheol Jung and Michael Taylor. “Evaluating Ruche Networks: Physically Scalable, Cost-Effective, Bandwidth-Flexible NoCs”. In: *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. ISCA ’25. Association for Computing Machinery, 2025, pp. 1035–1048. ISBN: 9798400712616. DOI: 10.1145/3695053.3731010. URL: <https://doi.org/10.1145/3695053.3731010>.
- [36] Jasmina Vasiljevic et al. “Compute Substrate for Software 2.0”. In: *IEEE Micro* 41.2 (2021), pp. 50–55. DOI: 10.1109/MM.2021.3061912.
- [37] David R. Ditzel and the Esperanto team. “Accelerating ML Recommendation With Over 1,000 RISC-V/Tensor Processors on Esperanto’s ET-SoC-1 Chip”. In: *IEEE Micro* 42.3 (2022), pp. 31–38. DOI: 10.1109/MM.2022.3140674.
- [38] Yichao Zhang et al. *TeraNoC: A Multi-Channel 32-bit Fine-Grained, Hybrid Mesh-Crossbar NoC for Efficient Scale-up of 1000+ Core Shared-L1-Memory Clusters*. 2025. arXiv: 2508.02446 [cs.DC]. URL: <https://arxiv.org/abs/2508.02446>.
- [39] Florian Zaruba et al. “Snitch: A Tiny Pseudo Dual-Issue Processor for Area and Energy Efficient Execution of Floating-Point Intensive Workloads”. In: *IEEE Transactions on Computers* 70.11 (Nov. 2021), pp. 1845–1860. ISSN: 1557-9956. DOI: 10.1109/TC.2020.3027900.
- [40] Samuel Riedel et al. “MemPool: A Scalable Manycore Architecture With a Low-Latency Shared L1 Memory”. In: *IEEE Transactions on Computers* 72.12 (2023), pp. 3561–3575. DOI: 10.1109/TC.2023.3307796.
- [41] Matteo Perotti et al. “Spatz: Clustering Compact RISC-V-Based Vector Units to Maximize Computing Efficiency”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44.7 (2025), pp. 2488–2502. DOI: 10.1109/TCAD.2025.3528349.
- [42] Chi Zhang. *Pulp-Platform-Insitu-Cache*. URL: <https://github.com/pulp-platform/Insitu-Cache>.
- [43] Lukas Steiner, Matthias Jung, Felipe S. Prado, et al. “DRAMSys4.0: An Open-Source Simulation Framework for In-depth DRAM Analyses”. In: *International Journal of Parallel Programming* 50 (April 2022 2022), pp. 217–242. DOI: 10.1007/s10766-022-00727-4. URL: <https://doi.org/10.1007/s10766-022-00727-4>.

A Private Write-Through L0.5 Cache Exploration

A.1 Motivation and Scope

The baseline CachePool architecture provides a logically shared L1 data cache that is physically distributed across Tiles and Groups. This organization avoids the need for a hardware cache-coherence protocol because cache lines are not replicated in independent private coherent copies. However, the physical location of a cache line still determines the interconnect distance required to access it. For workloads with strong per-core temporal locality, repeatedly accessing a remote shared-L1 bank can therefore incur unnecessary latency and interconnect traffic.

As a side exploration, we investigated the addition of a small private write-through cache between each core and the distributed shared L1. To keep the terminology of the main report unchanged, this additional level is referred to as the *L0.5 data cache*, while the existing distributed CachePool cache remains the L1.

Two approaches were explored. The first augments the private cache hierarchy with a hardware-managed directory-based coherence mechanism. The second retains the private write-through cache but removes hardware coherence and instead relies on explicit software synchronization and cache-management operations. The latter approach is the main focus of this appendix.

A.2 Hardware-Coherent Exploration

The first exploration introduced an HPDcache-based private data cache in front of the original shared CachePool data cache. The private cache operates in write-through mode, while the shared lower level remains logically shared and physically banked. A directory-based coherence mechanism was added to track private-cache ownership and sharers and to coordinate invalidations and ownership transitions.

This exploration demonstrated that a coherent private-cache hierarchy can be integrated into CachePool without fundamentally changing the surrounding scalar–vector organization. At the same time, the evaluation showed that the additional private-cache level introduces a non-negligible miss penalty, while coherence management adds further metadata accesses, arbitration, control flow, and coherence traffic. These observations motivated the exploration of a simpler alternative that retains the locality benefits of a private cache while eliminating the distributed hardware-coherence protocol.

The complete hardware-coherent design, including the cache organization, coherence protocol, message network, functional verification, performance analysis, and physical-design results, is documented in the accompanying Master’s thesis. **TODO: Add reference or appendix link to the attached Master**

A.3 Software-Managed Coherence

The second exploration retains the private write-through L0.5 cache but removes hardware-managed coherence. Each private cache operates independently, while the existing distributed shared L1 remains the common lower-level memory visible to all cores.

Write-through operation simplifies software-managed consistency because stores are propagated toward the shared L1 rather than retained indefinitely as dirty private copies. However, write-through alone is insufficient to guarantee coherence. A consumer may retain a stale private copy after another core updates the same shared data, while producer writes buffered inside the private cache may not yet have reached the shared L1 when synchronization occurs.

The explored design therefore couples synchronization with explicit cache-maintenance operations. At communication boundaries, software publishes modified data from the producer and invalidates potentially stale private copies at the consumer. This avoids hardware sharer tracking, coherence directories, and invalidation networks, at the cost of making data visibility an explicit software responsibility.

A.3.1 Private L0.5 Organization

The private L0.5 is based on HPDcache and is inserted between the Snitch–Spatz memory ports and the existing CachePool L1 interconnect. Both scalar and vector memory traffic therefore pass through the private cache before reaching the distributed shared L1.

The four Spatz memory lanes are coalesced into one wider HPDcache request, while the Snitch scalar memory port is presented to HPDcache as a separate requester. The corresponding response path scatters the vector response back to the individual Spatz lanes while returning scalar responses directly to Snitch. On the downstream side, HPDcache’s separate read and write memory channels are translated and merged back into the CachePool request/response protocol before entering the existing shared-cache interconnect.

The private cache is configured in write-through mode and does not maintain hardware coherence with other private caches. Consequently, ordinary cacheable data can be replicated and become stale across cores. Correctness is established only at explicit software synchronization boundaries.

Atomic memory operations are treated differently from ordinary cacheable accesses. AMOs are forwarded through the private level as uncacheable operations and are executed at the shared-cache side. This preserves a common serialization point for atomic synchronization even though ordinary data may be replicated in the private L0.5 caches.

A.3.2 Synchronization and Data Visibility

Software-managed coherence follows an acquire–release discipline around synchronization-protected shared data. On acquisition, a consuming core invalidates its private L0.5 so that subsequent reads cannot reuse stale lines and must obtain the current value from the shared hierarchy. Before releasing ownership, a producing core first waits for outstanding scalar and vector memory operations to complete and then drains the write-through buffer of its private cache.

Conceptually, a lock-protected critical section follows the sequence

```
acquire lock
lp1_inval()

    access protected shared data

snrt_fence()
lp1_wt_flush()
release lock
```

The invalidate operation provides the acquire-side visibility action by removing potentially stale private copies. The fence ensures that outstanding Snitch and Spatz memory operations complete before publication. The subsequent write-through flush drains buffered stores toward the shared L1 before the synchronization object is released.

This mechanism does not require cache maintenance for data that remain private to one core. Explicit maintenance is required only when data ownership or visibility crosses a synchronization boundary. The software overhead therefore depends on the communication structure of the workload rather than on the total number of memory accesses.

A.3.3 Runtime Cache-Management Operations

The software-managed scheme is supported by per-core Cache Management Operation (CMO) interfaces. Each core has an independent CMO injection path into its private cache so that cache maintenance does not serialize through one global control port.

The runtime exposes three principal operations:

- `lp1_wt_flush()` issues a write-through fence and drains the private-cache write buffer toward the shared L1;
- `lp1_inval()` invalidates the complete private L0.5 cache;
- `lp1_inval_line(addr)` invalidates only the cache line containing the selected address.

Each operation programs the calling core’s CMO control registers, commits the request, and waits until the private cache reports completion. The register interface is replicated per core so that independent cores can perform maintenance without contending for a shared trigger.

The implementation uses the historical prefix `lp1` for these functions and registers. In this report, the same private level is referred to as L0.5 to avoid renaming the existing shared CachePool L1.

A.3.4 Kernel Integration

The required cache-management sequence depends on how shared data are exchanged between cores. Private data require no additional operations, while data communicated through barriers or locks must be explicitly published by the producer and refreshed by the consumer.

For the FDOTP reduction, each producer completes its outstanding accesses and flushes the private write-through buffer before the synchronization point. The reducing core invalidates its private cache before reading the partial results so that the reduction observes the values published to the shared L1.

For spin-lock-based workloads, `lp1_inval()` is executed after lock acquisition and `snrt_fence()` followed by `lp1_wt_flush()` is executed before lock release. The same discipline is applied to multiple independent spin locks and to the MCS queue lock. These tests are particularly useful because repeated entry into the same critical section exposes stale private copies if the invalidate operation is omitted.

FFT requires cache-maintenance operations at inter-core communication boundaries between stages, while data that remain local to one core can stay resident in its private cache without additional maintenance. GEMV similarly applies explicit producer/consumer protection when results are exchanged between cores.

In addition to application kernels, dedicated tests were added for private-cache bring-up, capacity and eviction behavior, CMO liveness, atomic operations, LR/SC behavior, and cross-core synchronization. These tests provide functional coverage of the new private-cache datapath and the software-managed visibility mechanism.

A.4 Preliminary Performance Observations

Preliminary RTL experiments indicate that the usefulness of the private L0.5 is strongly workload dependent. Kernels with substantial temporal reuse can benefit from serving repeated accesses locally, while streaming kernels can suffer from the additional private-cache miss latency.

In an initial experiment on the `cachepool_fpu_16g_tiny` configuration with a 4 KiB private cache per core, FMATMUL showed a substantial increase in FPU utilization from 13.1% to 71.5%. GEMV improved from 9.0% to 17.4%. In contrast, FDOTP degraded because its low-reuse streaming access pattern repeatedly exercises the private-cache miss path.

Detailed waveform analysis indicates that a significant part of the FDOTP degradation originates from the HPDcache miss-handling pipeline. Directory lookup, MSHR allocation, refill processing, and replay can temporarily prevent new requests from entering the cache. The additional private level is therefore particularly costly when the hit rate is low and misses dominate execution.

These results suggest that the private L0.5 should be viewed as a workload-dependent optimization rather than a universally beneficial additional cache level. The potential gain is determined by the balance between local-hit reuse, miss penalty, synchronization frequency, and the cost of explicit cache-maintenance operations.

TODO: Consolidate the final performance evaluation with matched configurations and quantify the CMO

A.5 Discussion and Future Work

The software-managed approach substantially reduces hardware complexity compared with transparent cache coherence. It removes directory state, sharer tracking, coherence-message networks, and coherence transient states from the memory hierarchy. The corresponding cost is transferred to software, which must correctly identify communication boundaries and issue the required cache-maintenance operations.

This trade-off is particularly attractive for workloads whose sharing is structured around explicit barriers or locks. In these cases, synchronization points already exist and can also serve as data-visibility boundaries. The approach is less attractive for fine-grained or unsynchronized sharing, where frequent invalidation and write-buffer draining may eliminate the benefit of the private cache.

A broader design-space exploration is required to determine when the private L0.5 provides a net benefit. Relevant parameters include cache capacity and associativity, hit latency, miss penalty, write-buffer depth, CMO granularity, data-sharing frequency, and the physical distance to the corresponding shared-L1 banks.

A natural future step is to integrate the write-through L0.5 and its software-managed CMO behavior into the GVSoC framework described in Section 6. The higher simulation speed would allow cache size, topology, locality, and synchronization frequency to be explored over larger CachePool configurations and longer application workloads. Such an exploration could identify the workload classes and system scales for which reduced remote-access traffic outweighs the private-cache miss and maintenance overheads.

TODO: Implement the private L0.5 cache and CMO behavior in GVSoC if project time permits.

B CachePool Runtime API Reference

This appendix summarizes the user-facing functions provided by the CachePool runtime on the `dev/16ch` branch. The functions are grouped by purpose to complement the higher-level programming-model discussion in Section 5. Low-level register-pointer helpers and implementation-only interfaces are omitted, while low-level runtime functions that may be useful to application developers are retained for completeness.

Table 10: Summary of the user-facing CachePool runtime API. Functions are grouped by purpose; low-level register-pointer helpers and implementation-only interfaces are omitted.

Function	Scope	Description	Notes / Requirements
Topology and Identification			
<code>snrt_hartid()</code>	Per hart	Return the hardware hart identifier from <code>mhartid</code> .	Direct CSR read.
<code>snrt_global_core_base_hartid()</code>	System	Return the base hart ID of the current logical cluster.	Topology query.
<code>snrt_global_core_idx()</code>	Per core	Return the global core index.	System-wide identifier.
<code>snrt_global_core_num()</code>	System	Return the total number of cores.	Topology query.
<code>snrt_cluster_core_base_hartid()</code>	Cluster	Return the base hart ID of the current logical cluster.	Topology query.
<code>snrt_cluster_core_idx()</code>	Per core	Return the core index within the current logical cluster.	Common work-distribution index.
<code>snrt_cluster_core_num()</code>	Cluster	Return the number of cores in the current logical cluster.	Topology query.
<code>snrt_cluster_tile_idx()</code>	Per core	Return the Tile index of the executing core.	Cluster-wide Tile index.
<code>snrt_cluster_tile_num()</code>	Cluster	Return the number of Tiles in the current logical cluster.	Topology query.
<code>snrt_cluster_group_idx()</code>	Per core	Return the Group index of the executing core.	Cluster-wide Group index.
<code>snrt_cluster_group_num()</code>	Cluster	Return the number of Groups in the current logical cluster.	Topology query.
<code>snrt_cluster_core_per_tile()</code>	Cluster	Return the number of Snitch harts per Tile.	Depends on 1S1V/2S1V configuration.
<code>snrt_cluster_idx()</code>	Cluster	Return the logical-cluster index.	Relevant when multiple logical clusters are present.
<code>snrt_cluster_num()</code>	System	Return the number of logical clusters.	Topology query.
<code>snrt_cluster_is_primary()</code>	Per hart	Test whether the hart is host 0 of its shared-Spatz pair.	Always identifies the default owner in 2S1V.
<code>snrt_cluster_vpu_idx()</code>	Per VPU	Return the physical Spatz index.	Hart pairs map to one VPU in 2S1V.
<code>snrt_cluster_vpu_num()</code>	Cluster	Return the number of physical Spatz units.	Half the hart count in 2S1V.
<code>snrt_cluster_vpu_per_tile()</code>	Cluster	Return the number of Spatz units per Tile.	Configuration dependent.
<code>snrt_global_memory()</code>	System	Return the address slice describing global memory.	Returns <code>snrt_slice_t</code> .

Continued on next page

Table 10 – continued from previous page

Function	Scope	Description	Notes / Requirements
<code>snrt_cluster_memory()</code>	Cluster	Return the address slice associated with cluster-local memory metadata.	Returns <code>snrt_slice_t</code> .
Barrier Synchronization			
<code>snrt_cluster_hw_barrier()</code>	Cluster	Synchronize all cores using the hierarchical hardware barrier.	Uses slot 0 and full participation.
<code>snrt_cluster_sw_barrier()</code>	Cluster	Synchronize cores using the polling-based software barrier.	Software fallback.
<code>snrt_global_barrier()</code>	System	Synchronize across logical clusters.	Higher-level collective barrier.
<code>snrt_barrier(barr, n)</code>	Software	Execute a generic software barrier over <code>n</code> participants.	Uses a software barrier object.
<code>snrt_cluster_partial_barrier_mask(cids, n)</code>	Per core	Build a Tile-local participant mask from a list of global core IDs.	Result should be cached outside hot loops.
<code>snrt_cluster_partial_barrier(mask)</code>	Cluster	Restrict core participation within each Tile while retaining wider barrier scope.	Uses slot 0.
<code>snrt_cluster_group_barrier(core_mask, tile_mask, local_only, id)</code>	Group/System	Synchronize selected cores and Tiles on a chosen hardware-barrier slot.	All participants must use identical arguments.
<code>snrt_barrier_set_group_mask(id, mask)</code>	System	Configure the Group participation mask for a hardware-barrier slot.	Program from one core, then resynchronize before use.
<code>snrt_cluster_host0_barrier(id)</code>	Cluster	Synchronize only host 0 harts of each Core Complex.	Intended for 2S1V work sharing.
<code>snrt_cluster_host1_barrier(id)</code>	Cluster	Synchronize only host 1 harts of each Core Complex.	Intended for 2S1V work sharing.
Memory Ordering, Atomics, and Waiting			
<code>snrt_fence()</code>	Per hart	Drain outstanding Snitch and Spatz memory operations.	Use before synchronization or result readback when required.
<code>snrt_fence_snitch()</code>	Per hart	Drain outstanding Snitch scalar memory operations only.	Selective fence.
<code>snrt_fence_spatz()</code>	Per hart	Drain outstanding Spatz memory operations only.	Selective fence.

Continued on next page

Table 10 – continued from previous page

Function	Scope	Description	Notes / Requirements
<code>snrt_mutex_lock(pmtx)</code>	Shared memory	Acquire a blocking mutex using <code>amoswap.w.aq</code> .	Mutex word must be shared and initialized to zero.
<code>snrt_mutex_release(pmtx)</code>	Shared memory	Release a mutex using <code>amoswap.w.rl</code> .	Called by the lock owner.
<code>snrt_wfi()</code>	Per hart	Put the hart into wait-for-interrupt state.	Executes <code>wfi</code> .
L1 Address Mapping and Partitioning			
<code>l1d_xbar_config(offset)</code>	Cluster	Configure the L1 crossbar address-interleaving offset.	Granularity is clamped to at least one cache line.
<code>l1d_xbar_commit()</code>	Cluster/low level	Commit the programmed crossbar offset.	Normally used by the higher-level configuration path.
<code>l1d_part(size)</code>	Cluster	Set the number of Tile-private banks per Tile.	0=all shared; 4=all private in the standard configuration.
<code>l1d_part_folded(size, fold)</code>	Cluster	Set the private-bank count and choose Tile- or Group-private folding.	Shared banks remain globally shared.
<code>l1d_addr(addr)</code>	Cluster	Set the shared/private address boundary.	Current runtime maps addresses below the boundary shared and addresses at/above it private.
L1 Cache Maintenance			
<code>l1d_cluster_flush()</code>	Cluster	Flush all L1 banks across the cluster.	Called by all cores; runtime fences and synchronizes internally.
<code>l1d_cluster_shared_flush()</code>	Cluster	Flush only the globally shared L1 banks.	Called by all cores.
<code>l1d_cluster_private_flush(tile)</code>	Cluster	Flush private banks selected by the Tile mask.	Called by all cores.
<code>l1d_flush()</code>	Single core	Issue a low-level full L1 flush/invalidate operation.	Primarily for runtime/internal use.

Continued on next page

Table 10 – continued from previous page

Function	Scope	Description	Notes / Requirements
<code>l1d_shared_flush()</code>	Single core	Issue a low-level shared-bank flush.	Primarily for runtime/internal use.
<code>l1d_private_flush(tile)</code>	Single core	Issue a low-level private-bank flush for selected Tiles.	Primarily for runtime/internal use.
<code>l1d_wait()</code>	Single core	Poll until the current L1 maintenance operation completes.	Used by low-level maintenance functions.
<code>l1d_init(size)</code>	Boot/runtime	Initialize the L1 cache state.	Startup code performs cache invalidation before <code>main()</code> .
<code>l1d_commit()</code>	Cluster/low level	Commit the programmed L1 cache configuration.	Runtime helper.
Spatz Ownership Control			
<code>spatz_lock_acquire()</code>	Per hart	Acquire ownership of the shared Spatz unit.	Retries in software until accepted; relevant to 2S1V.
<code>spatz_lock_release()</code>	Per hart	Release ownership of the shared Spatz unit.	Must be called by the current owner.
<code>spatz_lock_try_acquire()</code>	Per hart	Perform one immediate ownership-acquire attempt.	Returns an encoded outcome without blocking in hardware.
<code>spatz_lock_try_release()</code>	Per hart	Perform one immediate ownership-release attempt.	Returns an encoded outcome.
<code>spatz_lock_outcome(raw)</code>	Local helper	Decode a raw Spatz-lock response into <code>FAIL</code> , <code>SUCCESS</code> , or <code>SUCCESS_WAIT</code> .	Inline helper.
Dynamic Memory Allocation and Data Movement			
<code>snrt_alloc_init(l3off)</code>	Single core	Initialize the DRAM heap starting at the configured offset.	Runtime initialization.
<code>snrt_malloc(size)</code>	Single core	Allocate a cache-line-aligned payload from the DRAM heap.	Not thread-safe; payload rounded to 64B.
<code>snrt_free(ptr)</code>	Single core	Free a DRAM allocation and coalesce adjacent free space when possible.	Not thread-safe.

Continued on next page

Table 10 – continued from previous page

Function	Scope	Description	Notes / Requirements
<code>snrt_memcpy(dst, src, n)</code>	Per core	Copy <code>n</code> bytes between memory regions.	Runtime-optimized memory copy.
Interrupt and Execution Support			
<code>snrt_int_init(team)</code>	Runtime	Initialize the interrupt subsystem for the current team.	Startup/runtime use.
<code>snrt_interrupt_global_enable()</code>	Per hart	Globally enable machine-mode interrupts.	Sets <code>mstatus.MIE</code> .
<code>snrt_interrupt_global_disable()</code>	Per hart	Globally disable machine-mode interrupts.	Clears <code>mstatus.MIE</code> .
<code>snrt_interrupt_enable(irq)</code>	Per hart	Enable an individual interrupt source.	Programs <code>mie</code> .
<code>snrt_interrupt_disable(irq)</code>	Per hart	Disable an individual interrupt source.	Programs <code>mie</code> .
<code>snrt_interrupt_cause()</code>	Per hart	Return the current interrupt cause code.	Derived from <code>mcause</code> .
<code>snrt_int_sw_set(hartid)</code>	System	Raise a software interrupt for a selected hart.	Low-level interrupt primitive.
<code>snrt_int_sw_clear(hartid)</code>	System	Clear a software interrupt for a selected hart.	Low-level interrupt primitive.
<code>snrt_int_sw_get(hartid)</code>	System	Read software-interrupt state for a selected hart.	Low-level interrupt primitive.
<code>snrt_int_sw_poll()</code>	Per hart	Poll for a software interrupt.	Low-level interrupt primitive.
<code>snrt_int_cluster_set(mask)</code>	Cluster	Raise software interrupts for selected cluster harts.	Bit-mask operation.
<code>snrt_int_cluster_clr(mask)</code>	Cluster	Clear software interrupts for selected cluster harts.	Bit-mask operation.
<code>snrt_int_clint_set(reg_off, mask)</code>	System	Write a CLINT interrupt-control mask at the selected register offset.	Low-level interrupt primitive.
<code>snrt_printf_float(val)</code>	Per hart	Print a single-precision floating-point value without promoting it to double.	Avoids unsupported compiler-generated double-precision operations.
<code>set_eoc(value)</code>	System control	Write the end-of-computation status value.	Used by runtime/test infrastructure.

C Spatz Supported Vector Instructions

The tables in this appendix summarize the vector instructions decoded by the `cachepool-32b` branch of Spatz. The decoder is used as the primary implementation-level reference. Instructions that remain only as legacy internal operation identifiers, but are not decoded by the current `spatz_decoder.sv`, are not listed as supported instructions. In particular, legacy MXU/matrix-related operation identifiers remain in `spatz_pkg.sv.tpl`, but no corresponding matrix instruction is decoded by the current decoder and they are therefore omitted here.

The `cachepool-32b` implementation has a maximum vector element width of 32 bits unless double-precision support is enabled. The decoder still recognizes 64-bit vector memory encodings, but explicitly marks them illegal when the configured maximum element width is below 64 bits. Vector floating-point instructions are available only when Spatz is configured with floating-point support. The Spatz-specific widening dot-product instructions are listed separately from standard RVV instructions.

Table 11: Integer, memory, reduction, and permutation instructions supported by the Spatz `cachepool-32b` decoder.

Class	Operation	Supported instruction forms	Condition
Vector memory operations			
Memory	Unit-stride load	<code>vle8.v, vle16.v, vle32.v, vle64.v</code>	64-bit width requires MAXEW=EW_64
	Strided load	<code>vlse8.v, vlse16.v, vlse32.v, vlse64.v</code>	64-bit width requires MAXEW=EW_64
	Unordered indexed load	<code>vluxei8.v, vluxei16.v, vluxei32.v, vluxei64.v</code>	64-bit index width requires MAXEW=EW_64
	Ordered indexed load	<code>vloxei8.v, vloxei16.v, vloxei32.v, vloxei64.v</code>	64-bit index width requires MAXEW=EW_64
	Unit-stride store	<code>vse8.v, vse16.v, vse32.v, vse64.v</code>	64-bit width requires MAXEW=EW_64
	Strided store	<code>vsse8.v, vsse16.v, vsse32.v, vsse64.v</code>	64-bit width requires MAXEW=EW_64

Class	Operation	Supported instruction forms	Condition
	Unordered indexed store	<code>vsuxei8.v</code> , <code>vsuxei16.v</code> , <code>vsuxei32.v</code> , <code>vsuxei64.v</code>	64-bit index width requires <code>MAXEW=EW_64</code>
	Ordered indexed store	<code>vsoxei8.v</code> , <code>vsoxei16.v</code> , <code>vsoxei32.v</code> , <code>vsoxei64.v</code>	64-bit index width requires <code>MAXEW=EW_64</code>
Integer arithmetic and logic			
Integer arithmetic	Add	<code>vadd.vv</code> , <code>vadd.vx</code> , <code>vadd.vi</code>	—
	Subtract	<code>vsub.vv</code> , <code>vsub.vx</code>	—
	Reverse subtract	<code>vrsb.vx</code> , <code>vrsb.vi</code>	—
Integer logic	AND	<code>vand.vv</code> , <code>vand.vx</code> , <code>vand.vi</code>	—
	OR	<code>vor.vv</code> , <code>vor.vx</code> , <code>vor.vi</code>	—
	XOR	<code>vxor.vv</code> , <code>vxor.vx</code> , <code>vxor.vi</code>	—
Carry/borrow	Add with carry	<code>vadc.vvm</code> , <code>vadc.vxm</code> , <code>vadc.vim</code>	—
	Add with carry-out	<code>vmadc.vv</code> , <code>vmadc.vx</code> , <code>vmadc.vi</code> , <code>vmadc.vvm</code> , <code>vmadc.vxm</code> , <code>vmadc.vim</code>	—
	Subtract with borrow	<code>vsbc.vvm</code> , <code>vsbc.vxm</code>	—
	Subtract with borrow-out	<code>vmsbc.vv</code> , <code>vmsbc.vx</code> , <code>vmsbc.vvm</code> , <code>vmsbc.vxm</code>	—
Shifts	Logical left	<code>vsll.vv</code> , <code>vsll.vx</code> , <code>vsll.vi</code>	—
	Logical right	<code>vsrl.vv</code> , <code>vsrl.vx</code> , <code>vsrl.vi</code>	—
	Arithmetic right	<code>vsra.vv</code> , <code>vsra.vx</code> , <code>vsra.vi</code>	—
Min/max	Signed minimum	<code>vmin.vv</code> , <code>vmin.vx</code>	—
	Unsigned minimum	<code>vminu.vv</code> , <code>vminu.vx</code>	—
	Signed maximum	<code>vmax.vv</code> , <code>vmax.vx</code>	—
	Unsigned maximum	<code>vmaxu.vv</code> , <code>vmaxu.vx</code>	—
Integer comparisons			
Comparison	Equal	<code>vmseq.vv</code> , <code>vmseq.vx</code> , <code>vmseq.vi</code>	—
	Not equal	<code>vmsne.vv</code> , <code>vmsne.vx</code> , <code>vmsne.vi</code>	—
	Unsigned less-than	<code>vmsltu.vv</code> , <code>vmsltu.vx</code>	—
	Signed less-than	<code>vmslt.vv</code> , <code>vmslt.vx</code>	—
	Unsigned less-or-equal	<code>vmsleu.vv</code> , <code>vmsleu.vx</code> , <code>vmsleu.vi</code>	—

Class	Operation	Supported instruction forms	Condition
	Signed less-or-equal	<code>vmsle.vv</code> , <code>vmsle.vx</code> , <code>vmsle.vi</code>	—
	Unsigned greater-than	<code>vmsgtu.vx</code> , <code>vmsgtu.vi</code>	—
	Signed greater-than	<code>vmsgt.vx</code> , <code>vmsgt.vi</code>	—
Integer multiply, divide, and multiply-accumulate			
Multiply	Low product	<code>vmul.vv</code> , <code>vmul.vx</code>	—
	High signed product	<code>vmulh.vv</code> , <code>vmulh.vx</code>	—
	High unsigned product	<code>vmulhu.vv</code> , <code>vmulhu.vx</code>	—
	High signed–unsigned product	<code>vmulhsu.vv</code> , <code>vmulhsu.vx</code>	—
Widening multiply	Signed	<code>vwmul.vv</code> , <code>vwmul.vx</code>	—
	Unsigned	<code>vwmulu.vv</code> , <code>vwmulu.vx</code>	—
	Signed–unsigned	<code>vwmulsu.vv</code> , <code>vwmulsu.vx</code>	—
Divide/remainder	Unsigned divide	<code>vdivu.vv</code> , <code>vdivu.vx</code>	—
	Signed divide	<code>vdiv.vv</code> , <code>vdiv.vx</code>	—
	Unsigned remainder	<code>vremu.vv</code> , <code>vremu.vx</code>	—
	Signed remainder	<code>vrem.vv</code> , <code>vrem.vx</code>	—
Multiply-add	Multiply-accumulate	<code>vmacc.vv</code> , <code>vmacc.vx</code>	—
	Negated multiply-subtract accumulate	<code>vnmsac.vv</code> , <code>vnmsac.vx</code>	—
	Multiply-add	<code>vmadd.vv</code> , <code>vmadd.vx</code>	—
Widening multiply-add	Negated multiply-subtract	<code>vnmsub.vv</code> , <code>vnmsub.vx</code>	—
	Signed accumulate	<code>vwmacc.vv</code> , <code>vwmacc.vx</code>	—
	Unsigned accumulate	<code>vwmaccu.vv</code> , <code>vwmaccu.vx</code>	—
	Signed–unsigned accumulate	<code>vwmaccsu.vv</code> , <code>vwmaccsu.vx</code>	—
	Unsigned–signed accumulate	<code>vwmaccus.vx</code>	—
Integer reductions			
Reduction	Sum	<code>vredsum.vs</code>	—
	AND	<code>vredand.vs</code>	—
	OR	<code>vredor.vs</code>	—
	XOR	<code>vredxor.vs</code>	—

Class	Operation	Supported instruction forms	Condition
	Signed minimum	<code>vredmin.vs</code>	—
	Unsigned minimum	<code>vredminu.vs</code>	—
	Signed maximum	<code>vredmax.vs</code>	—
	Unsigned maximum	<code>vredmaxu.vs</code>	—
Moves, merge, and slide operations			
Merge	Masked merge	<code>vmerge.vvm, vmerge.vxm, vmerge.vim</code>	—
Move	Vector/scalar/immediate move	<code>vmv.v.v, vmv.v.x, vmv.v.i</code>	—
	Scalar to vector element	<code>vmv.s.x</code>	—
	Vector element to scalar	<code>vmv.x.s</code>	—
Slide	Slide up	<code>vslideup.vx, vslideup.vi</code>	—
	Slide one up	<code>vslide1up.vx</code>	—
	Slide down	<code>vslidedown.vx, vslidedown.vi</code>	—
	Slide one down	<code>vslide1down.vx</code>	—
Vector configuration and CSR access			
Configuration	Set vector length/type	<code>vsetvl, vsetvli, vsetivli</code>	—
CSR access	Vector CSR access through standard CSR instructions	<code>csrrw, csrrs, csrrc, csrrwi, csrrsi, csrrci</code>	Only vector CSRs are accepted

Table 12: Floating-point and Spatz-specific vector instructions supported by the Spatz cachepool-32b decoder.

Class	Operation	Supported instruction forms	Condition
Basic floating-point arithmetic			
Floating-point arithmetic	Add	vfadd.vv, vfadd.vf	FPU
	Subtract	vfsb.vv, vfsb.vf	FPU
	Reverse subtract	vfrsub.vf	FPU
	Minimum	vfmin.vv, vfmin.vf	FPU
	Maximum	vfmax.vv, vfmax.vf	FPU
	Sign inject	vfsgnj.vv, vfsgnj.vf	FPU
	Sign inject negated	vfsgnjn.vv, vfsgnjn.vf	FPU
	Sign inject XOR	vfsgnjx.vv, vfsgnjx.vf	FPU
	Multiply	vfmul.vv, vfmul.vf	FPU
Floating-point fused multiply-add			
Fused multiply-add	Multiply-add	vfmadd.vv, vfmadd.vf	FPU
	Negated multiply-add	vfnmadd.vv, vfnmadd.vf	FPU
	Multiply-subtract	vfmsub.vv, vfmsub.vf	FPU
	Negated multiply-subtract	vfnmsub.vv, vfnmsub.vf	FPU
	Multiply-accumulate	vfmac.vv, vfmac.vf	FPU
	Negated multiply-accumulate	vfnmac.vv, vfnmac.vf	FPU
	Multiply-subtract accumulate	vfmsac.vv, vfmsac.vf	FPU
	Negated multiply-subtract accumulate	vfnmsac.vv, vfnmsac.vf	FPU
Floating-point reductions			
Reduction	Ordered sum	vfredosum.vs	FPU
	Unordered sum	vfredusum.vs	FPU
	Minimum	vfredmin.vs	FPU
	Maximum	vfredmax.vs	FPU
Floating-point conversions and moves			

Class	Operation	Supported instruction forms	Condition
Conversion	Signed integer to floating point	<code>vfcvt.f.x.v</code>	FPU
	Unsigned integer to floating point	<code>vfcvt.f.xu.v</code>	FPU
	Floating point to signed integer	<code>vfcvt.x.f.v</code> , <code>vfcvt.rtz.x.f.v</code>	FPU
	Floating point to unsigned integer	<code>vfcvt.xu.f.v</code> , <code>vfcvt.rtz.xu.f.v</code>	FPU
Narrowing conversion	Floating point to unsigned integer	<code>vfncvt.xu.f.w</code> , <code>vfncvt.rtz.xu.f.w</code>	FPU
	Floating point to signed integer	<code>vfncvt.x.f.w</code> , <code>vfncvt.rtz.x.f.w</code>	FPU
	Unsigned integer to floating point	<code>vfncvt.f.xu.w</code>	FPU
	Signed integer to floating point	<code>vfncvt.f.x.w</code>	FPU
	Floating point to narrower floating point	<code>vfncvt.f.f.w</code>	FPU
Move	Scalar FP to vector	<code>vfmv.v.f</code> , <code>vfmv.s.f</code>	FPU
	Vector FP element to scalar FP	<code>vfmv.f.s</code>	FPU
Widening floating-point operations			
Widening add	Vector/vector and wide-vector/vector	<code>vfwadd.vv</code> , <code>vfwadd.wv</code>	FPU
	Vector/scalar and wide-vector/scalar	<code>vfwadd.vf</code> , <code>vfwadd.wf</code>	FPU
Widening subtract	Vector/vector and wide-vector/vector	<code>vfwsb.vv</code> , <code>vfwsb.wv</code>	FPU
	Vector/scalar and wide-vector/scalar	<code>vfwsb.vf</code> , <code>vfwsb.wf</code>	FPU
Widening multiply	Multiply	<code>vfwmul.vv</code> , <code>vfwmul.vf</code>	FPU
Widening fused multiply-add	Multiply-accumulate	<code>vfwmac.vv</code> , <code>vfwmac.vf</code>	FPU

Class	Operation	Supported instruction forms	Condition
	Negated multiply-accumulate	<code>vfnmacc.vv</code> , <code>vfnmacc.vf</code>	FPU
	Multiply-subtract accumulate	<code>vfwmsac.vv</code> , <code>vfwmsac.vf</code>	FPU
	Negated multiply-subtract accumulate	<code>vfnmsac.vv</code> , <code>vfnmsac.vf</code>	FPU
Floating-point slide operations			
Slide	Slide one up with FP scalar	<code>vfslide1up.vf</code>	FPU
	Slide one down with FP scalar	<code>vfslide1down.vf</code>	FPU
Spatz-specific extension			
Dot product	Widening floating-point dot product	<code>vfwdotp.vv</code> , <code>vfwdotp.vf</code>	FPU; custom instruction